



Introduction to Caching

Agenda

- What is caching?
- Classical System Model
- Basic Caching Algorithms
- Cost Aware Caching Algorithms
- Adaptive Caching Algorithms





What is Caching?

Definition

Caching is the process of storing a *part* of the available data in a medium capable of serving requests *faster* (with a lower latency) or *with less resources* (less CPU usage, less bandwidth usage, etc.)

The cache is a component with limited space, thus, to use a cache, we need to define:

An *Admission Policy*

An *Eviction Policy*

Typically, the evicted item is called the *victim* of the cache.

Caching types



- Caching is divided between two domains:
- Hardware Level Caching
 - CPU
 - Storage Devices
- Software Level Caching
 - Servers
 - Services

Hardware Level Caches

- Implementing very simple Admission and Eviction policies due to hardware limitation and costs.
- Based on very fast mediums but extremely size-limited
 - Recent improvements in CPU performance is due to increased L1 to L3 cache sizes
 - Common in Hard Disk Drives (HDD), using a small ephemeral storage (usually flash drives)

Software Level Caches

- Less constrained by hardware, however, constant time operations are expected
- Used in variety of different applications:
 - Distributed Data Storage Systems
 - Content Delivery Networks
 - Databases
 - Network based applications
 - Machine Learning Algorithms

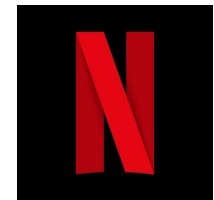
Software Level Caches

- Less constrained by hardware, however, constant time operations are expected
- Used in variety of different applications:
 - Distributed Data Storage Systems
 - Content Delivery Networks
 - Databases
 - Network based applications
 - Machine Learning Algorithms



Software Level Caches

- Less constrained by hardware, however, constant time operations are expected
- Used in variety of different applications:
 - Distributed Data Storage Systems
 - Content Delivery Networks
 - Databases
 - Network based applications
 - Machine Learning Algorithms



Software Level Caches

- Less constrained by hardware, however, constant time operations are expected
- Used in variety of different applications:
 - Distributed Data Storage Systems
 - Content Delivery Networks
 - Databases
 - Network based applications
 - Machine Learning Algorithms



Software Level Caches

- Less constrained by hardware, however, constant time operations are expected
- Used in variety of different applications:
 - Distributed Data Storage Systems
 - Content Delivery Networks
 - Databases
 - Network based applications
 - Machine Learning Algorithms



Software Level Caches



The idea is holding responses for queries at a closer location

- On network level – on a closer server
- On machine level – on the main memory (RAM), on faster disks (Solid State Disks)
- Algorithms - When the computation is expensive

The major focus is on *eviction policies*

A black and white photograph of a young plant seedling with two leaves emerging from dark, textured soil. The seedling is positioned on the left side of the frame. The background is dark and out of focus.

The Classical Model

Definitions



Our workflow is a sequence of requests ordered by their arrival times (no simultaneous requests).

Each request is for an *item*.

All items are *unit-sized*.

Each request may be:

- Served directly from the cache – a *Cache Hit*

- Requested from the data source – a *Cache Miss*

The cache *Hit Ratio* is considered the optimization metric of the cache.

A proxy of the Average Latency experienced by the users.

Example

Cache

The Hit Ratio is: $\frac{9}{13} \approx 69\%$

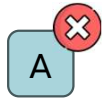
Example

A

Cache

The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



Cache

The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



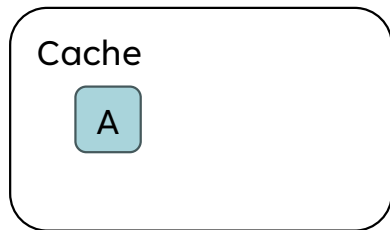
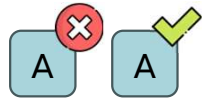
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



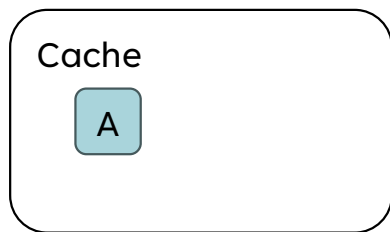
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



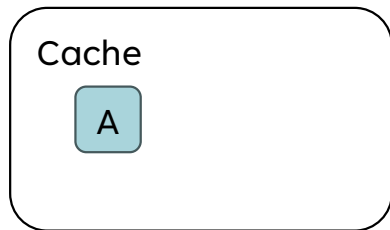
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



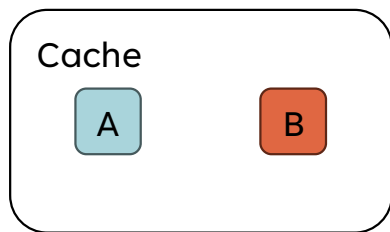
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



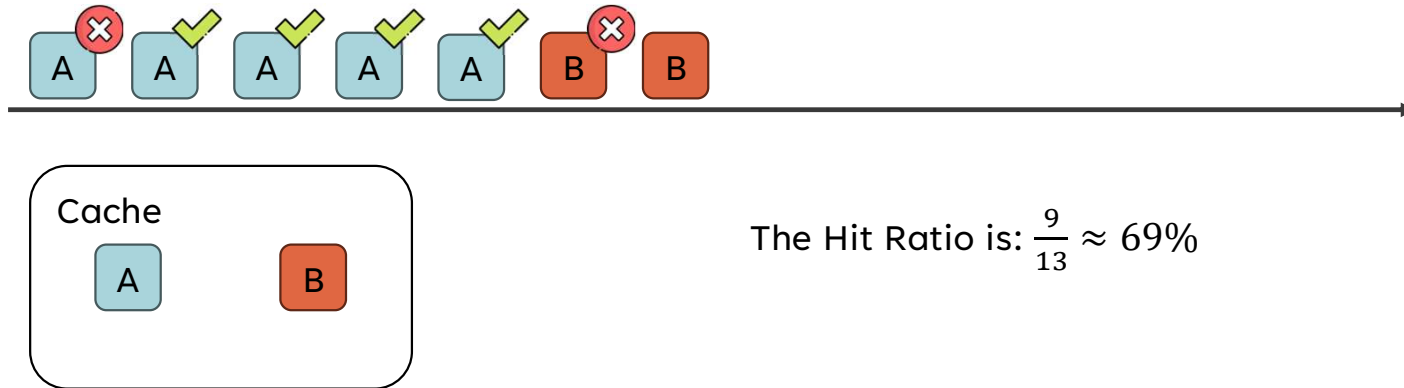
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



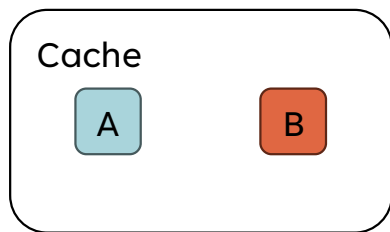
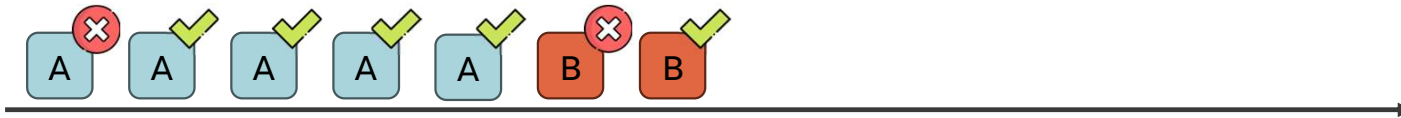
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



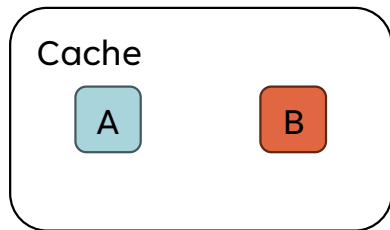
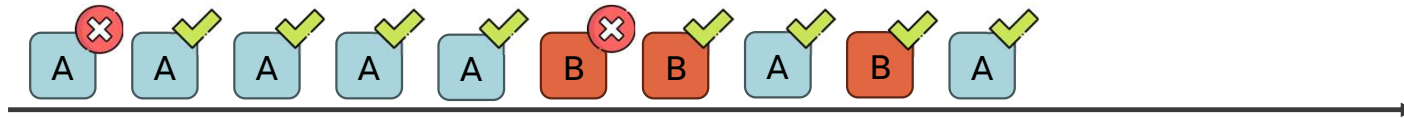
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



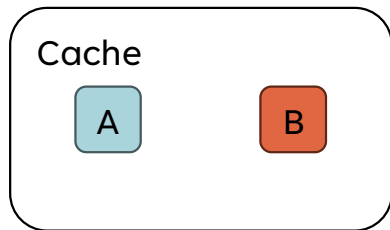
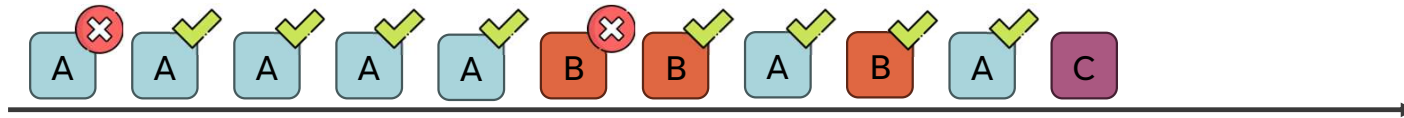
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



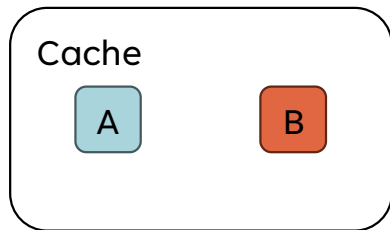
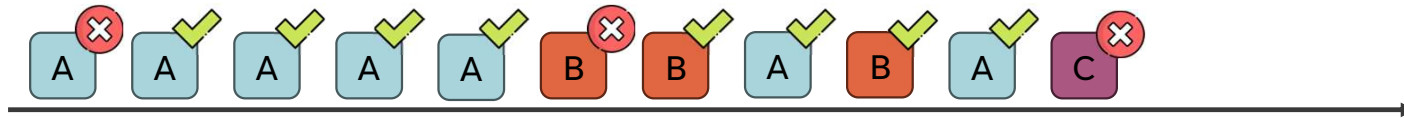
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



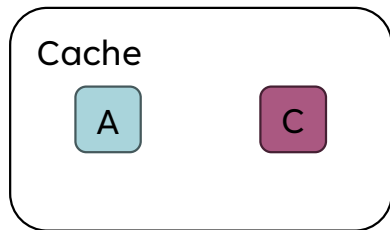
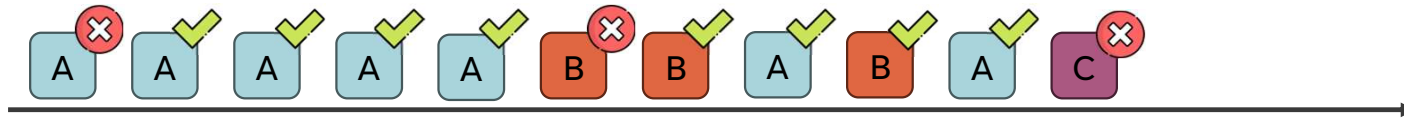
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



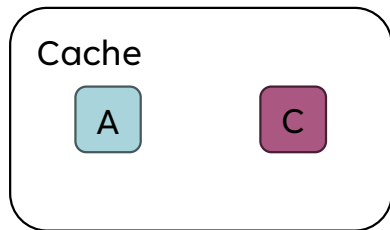
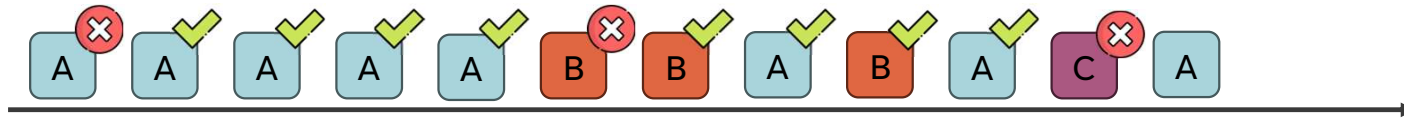
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



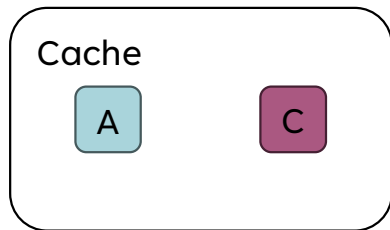
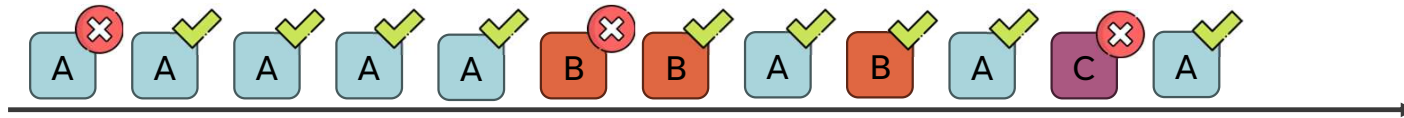
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



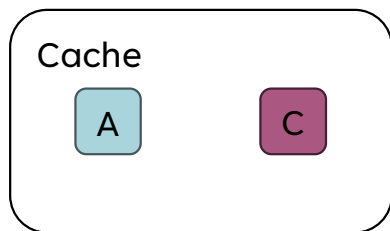
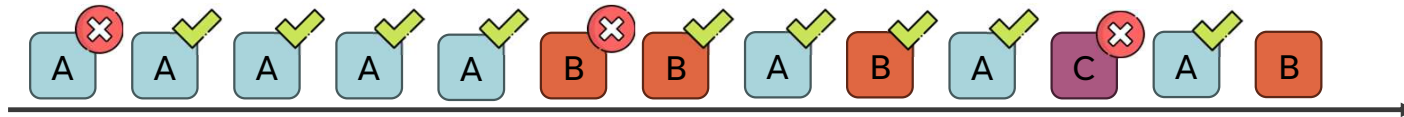
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



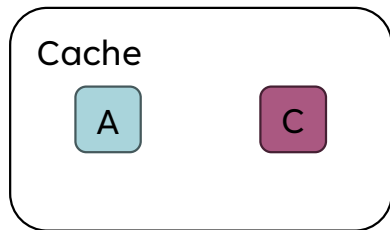
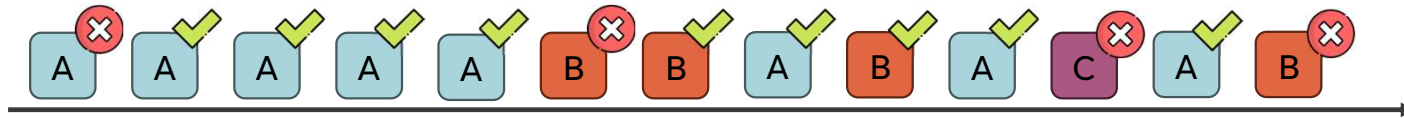
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



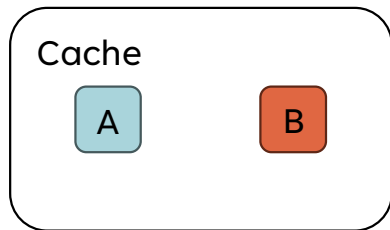
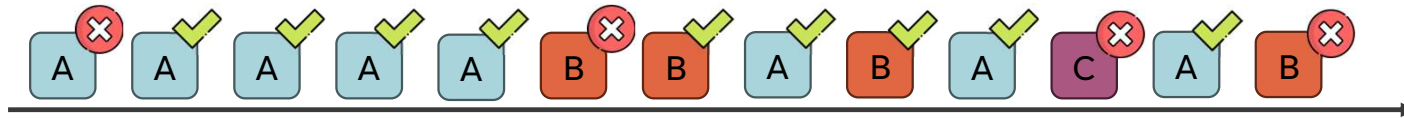
The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Example



The Hit Ratio is: $\frac{9}{13} \approx 69\%$

Few Important Notes



Not all traffic can be served –

One Hit Wonders are an issue of all caching strategies.

Not all workflows are similar –

Different applications (usually) require different solutions.

The items are comparably large (caching is not very efficient on byte-level items).

It is difficult to analyze the traffic before deploying a cache.

Reading information from RAM is **orders of magnitude** faster than reading it from disk and fetching information from a different source is even worse!

The majority of the run-time of most programs is due to I/O operations!

A black and white photograph of a small seedling with two leaves growing out of dark, textured soil. The seedling is positioned on the left side of the frame, and the soil is in the foreground. The background is dark and out of focus.

Basic Caching Algorithms

Basic Caching Algorithms Design



- As mentioned before, the policies are divided into two groups:
 - Admission Policies – Do not admit One-Hit-Wonders (how?), do not admit items larger than X, etc.
 - Eviction Policies – Decide what is the *least important* item in the cache and evict it.
 - The importance of an item is a score given by the policy, according to a certain characteristic of the item, or its access pattern.
 - With no time complexity considerations, we may rank all the items in the cache and evict the lowest scored item.
 - In practice, we want to perform only $\Theta(1)$ time operations, with $\Theta(\log n)$ being borderline usable.

Common Caching Policies

- **Single Access Pattern Policies**

- Belady Clairvoyant Policy
- First-In First-Out
- Least Recently Used
- Least Frequently Used
- Low Inter-arrival Recency Set

- **Combination of Access Patterns Policies**

- Segmented Least Recently Used
- Two-Queue
- Least Recently / Frequently Used
- Window Tiny-LFU





Single Access Pattern Policies

Belady's Clairvoyant Policy

- The optimal algorithm assuming the entire sequence is known **in advance**.
- Used as a benchmark for on-line algorithms.
- Not a practical solution.
- The victim of the cache is the item whose next access is the **farthest away** in the future.

First-In First-Out

- The items are arranged in a simple queue, thus very efficient
- The victim is the item that was inserted first
- Some variations on the idea exists:
 - Second-Chance
 - Clock

Least Recently Used

The *LRU* policy prioritizes items according to their *recency*.

This policy always admits a new item upon request.

The victim of the cache is the item that was last used least recently.

How?

Used in many applications due to its simplicity.

The most well-known caching algorithm.

Least Recently Used

LRU Cache

Head
→

Hottest ← → Coldest

Least Recently Used

A

LRU Cache

Head
→

Hottest ← → Coldest

Least Recently Used

A

LRU Cache

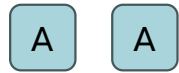
Head

A

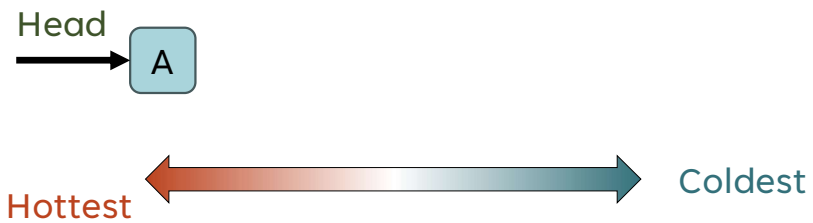
Hottest

Coldest

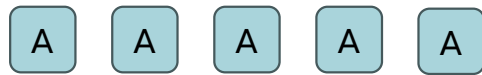
Least Recently Used



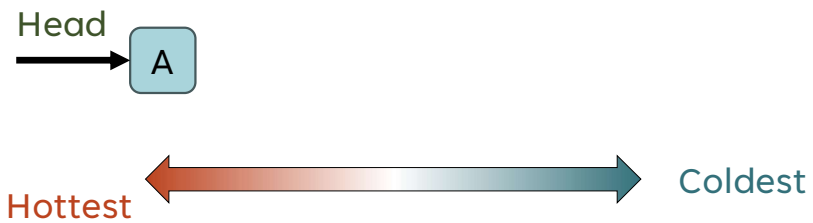
LRU Cache



Least Recently Used



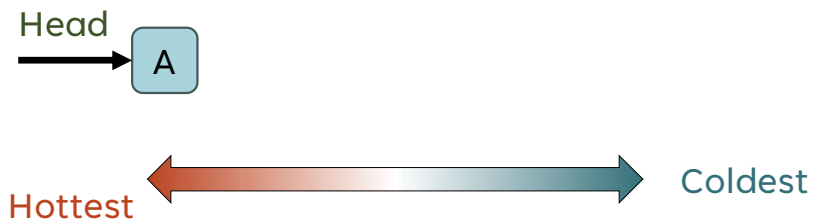
LRU Cache



Least Recently Used



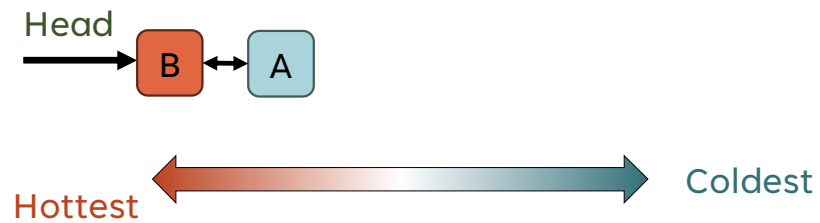
LRU Cache



Least Recently Used



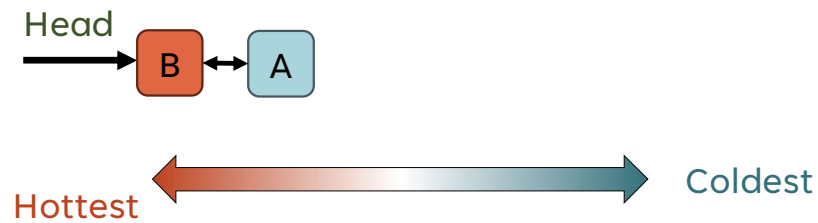
LRU Cache



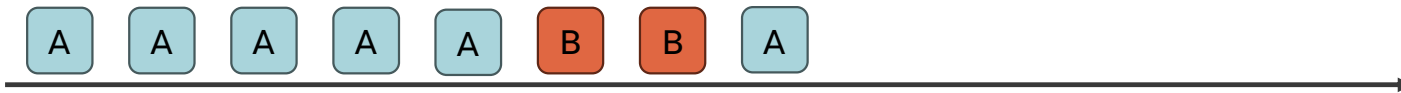
Least Recently Used



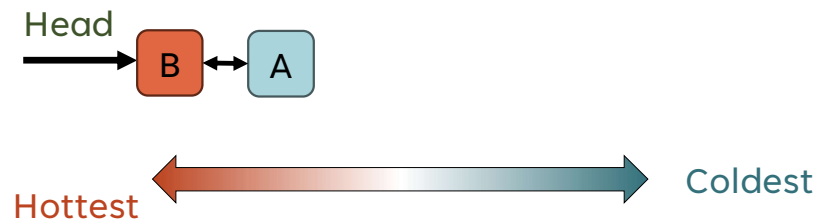
LRU Cache



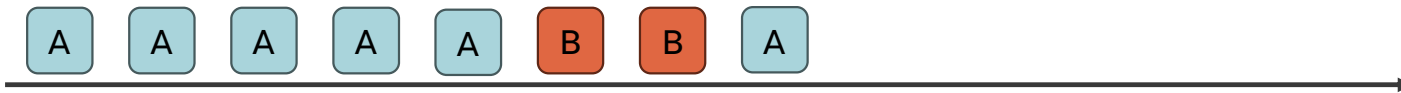
Least Recently Used



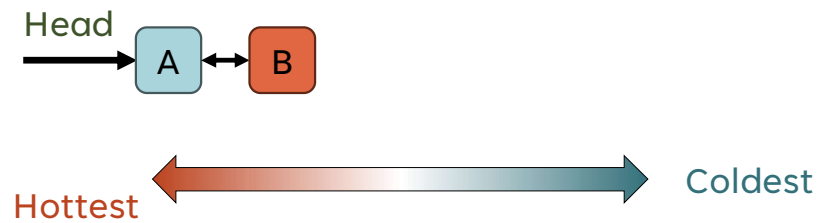
LRU Cache



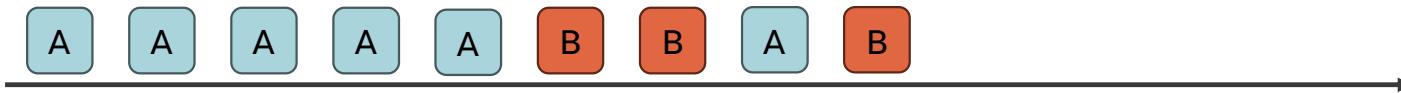
Least Recently Used



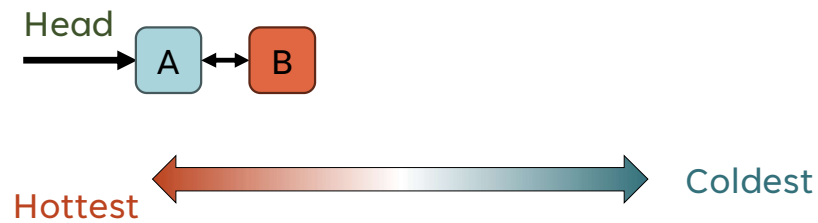
LRU Cache



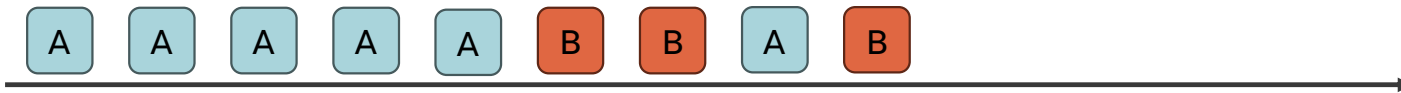
Least Recently Used



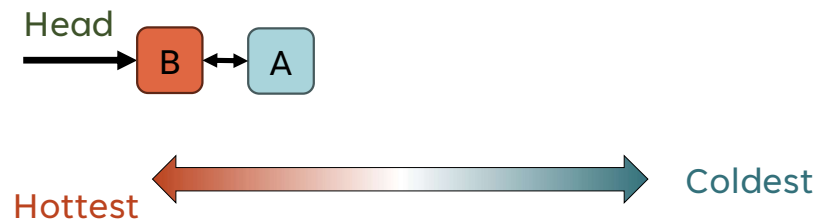
LRU Cache



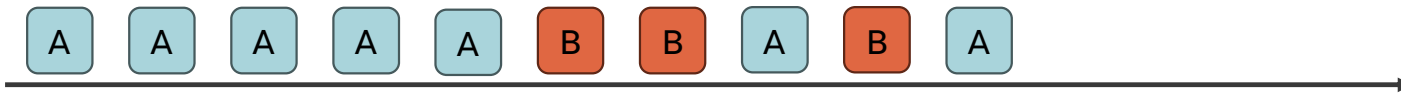
Least Recently Used



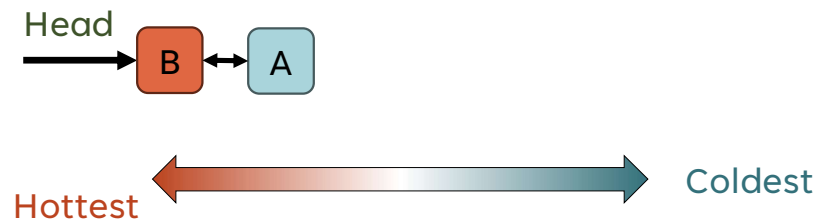
LRU Cache



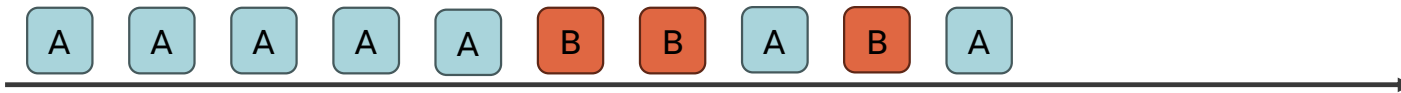
Least Recently Used



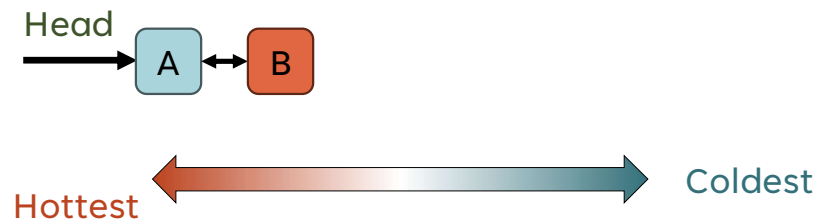
LRU Cache



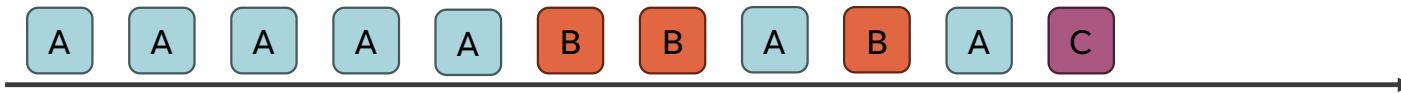
Least Recently Used



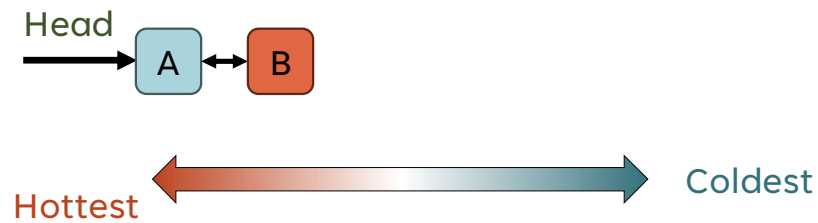
LRU Cache



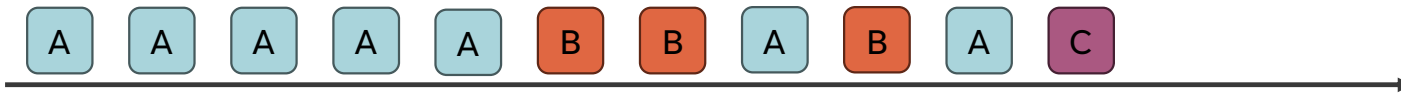
Least Recently Used



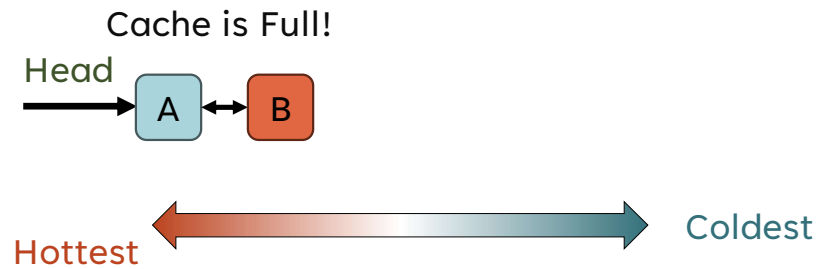
LRU Cache



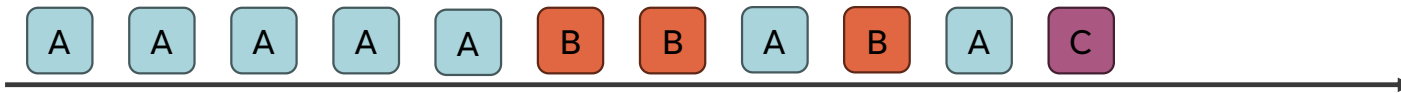
Least Recently Used



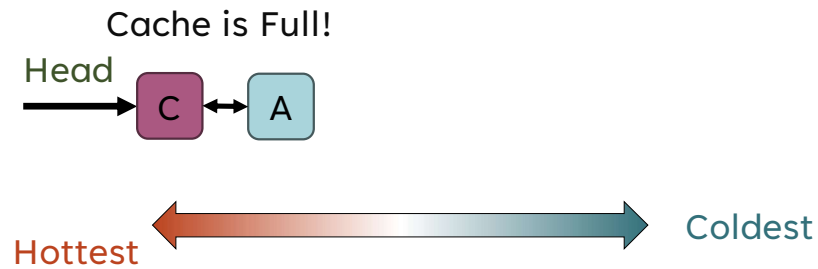
LRU Cache



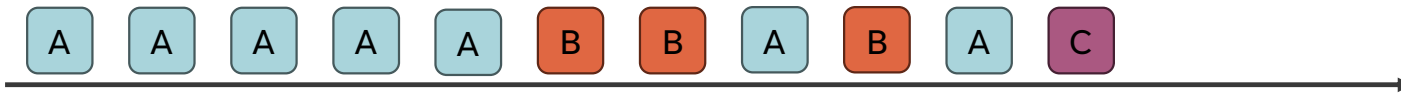
Least Recently Used



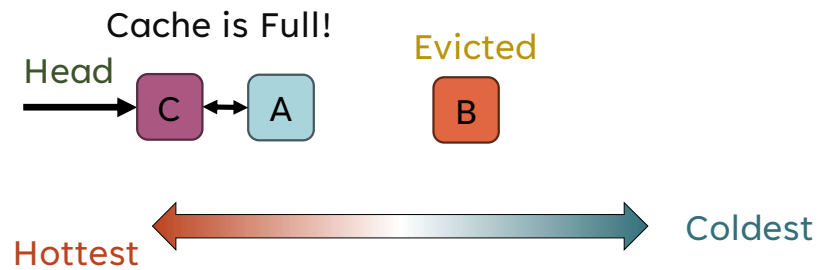
LRU Cache



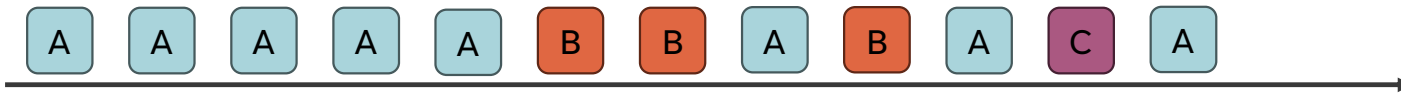
Least Recently Used



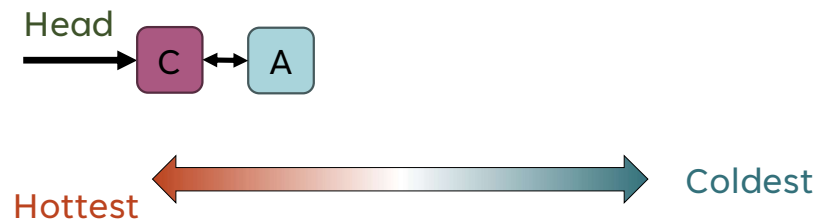
LRU Cache



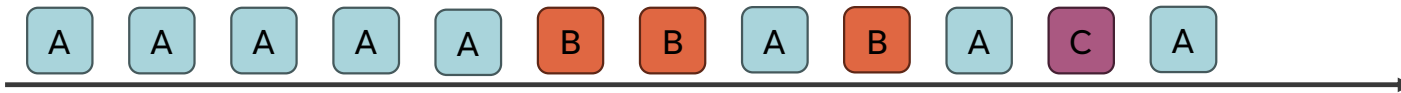
Least Recently Used



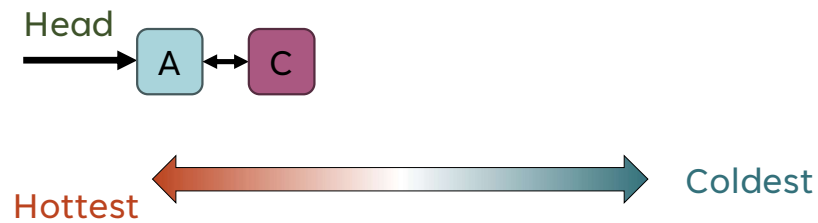
LRU Cache



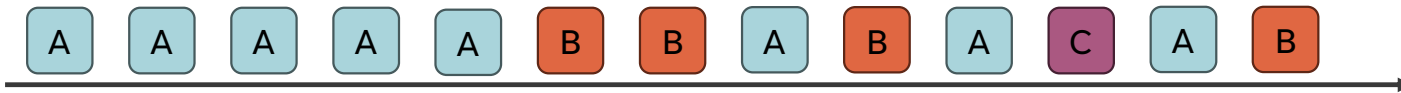
Least Recently Used



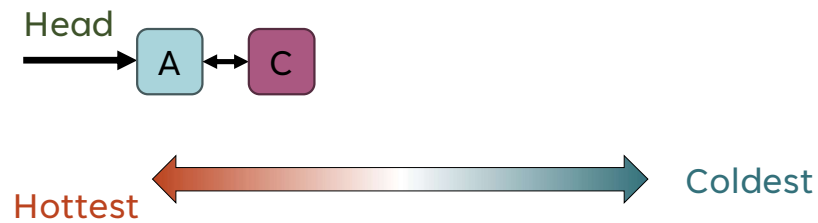
LRU Cache



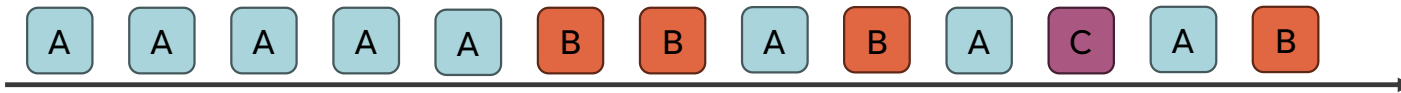
Least Recently Used



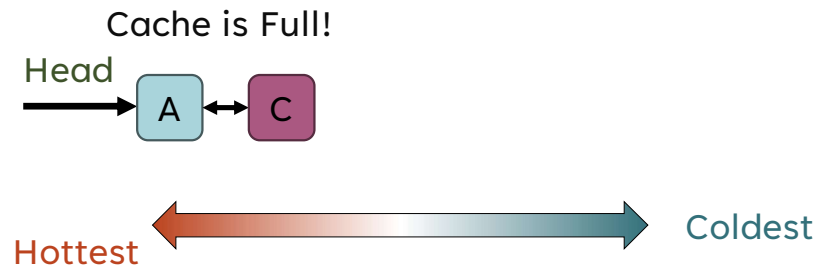
LRU Cache



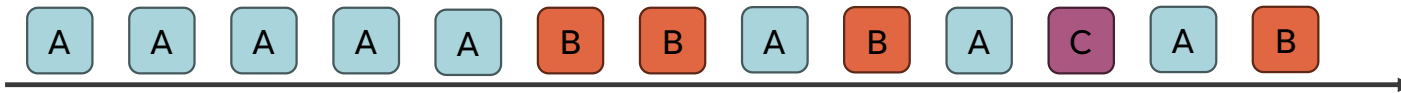
Least Recently Used



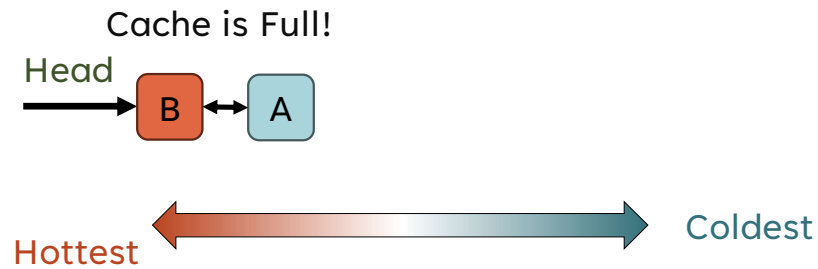
LRU Cache



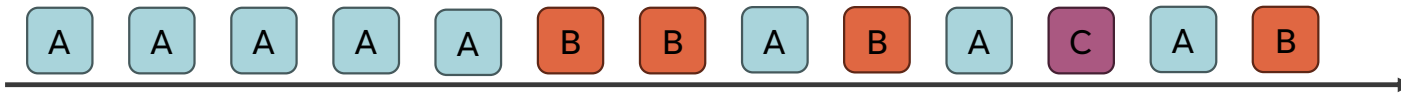
Least Recently Used



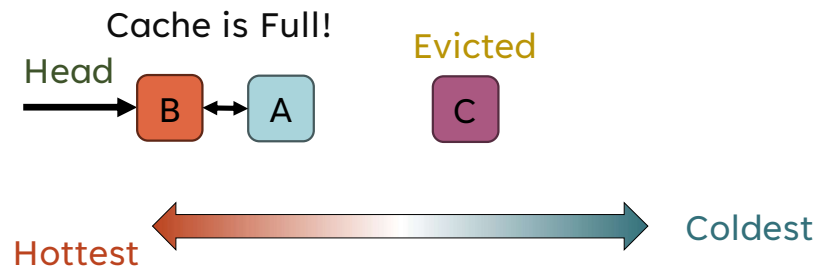
LRU Cache



Least Recently Used



LRU Cache



Least Frequently Used

The *LFU* policy priorities items according to their *frequency*.

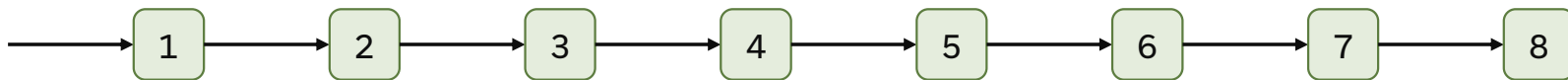
This policy may prefer not to admit a new item.

The victim of the cache is the item that was queried the least times.

How?

Least Frequently Used

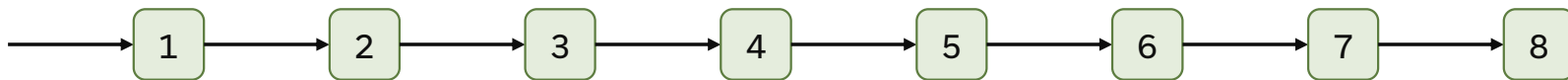
LFU Cache



Least Frequently Used

A

LFU Cache

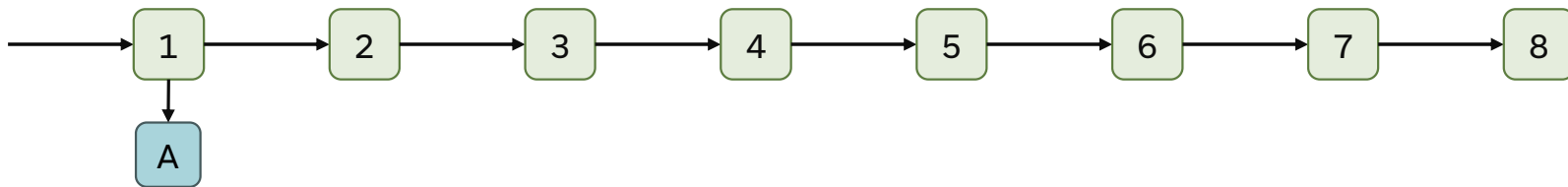


Least Frequently Used

A



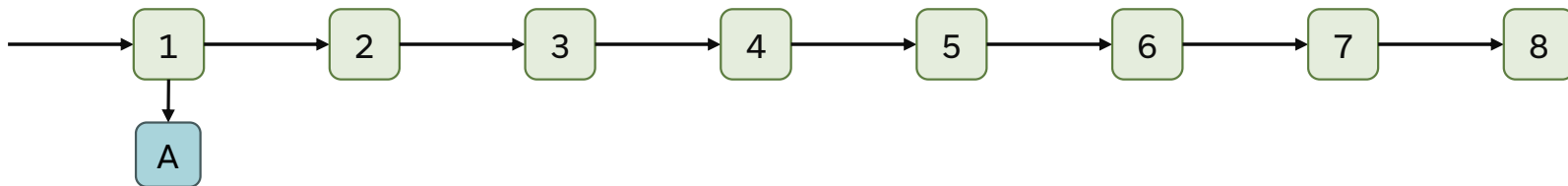
LFU Cache



Least Frequently Used



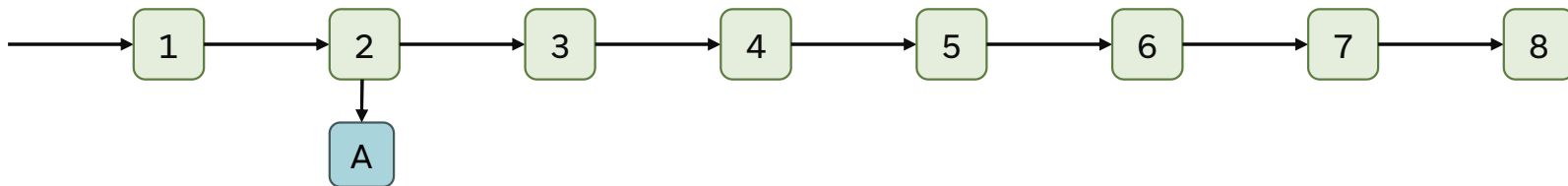
LFU Cache



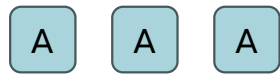
Least Frequently Used



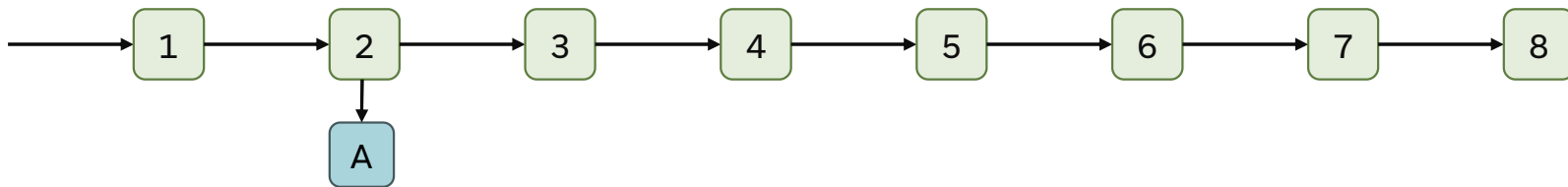
LFU Cache



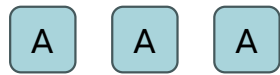
Least Frequently Used



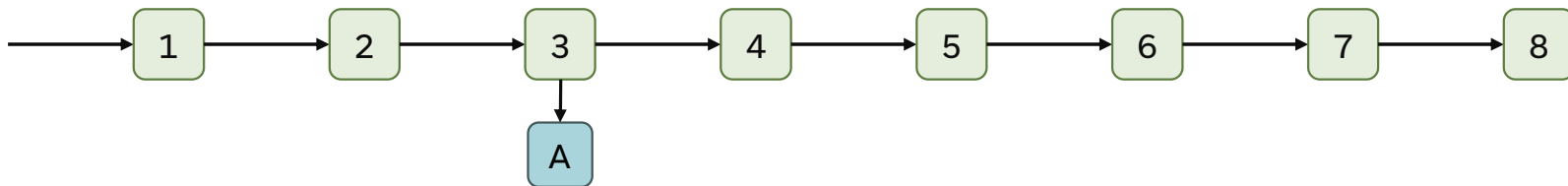
LFU Cache



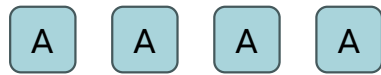
Least Frequently Used



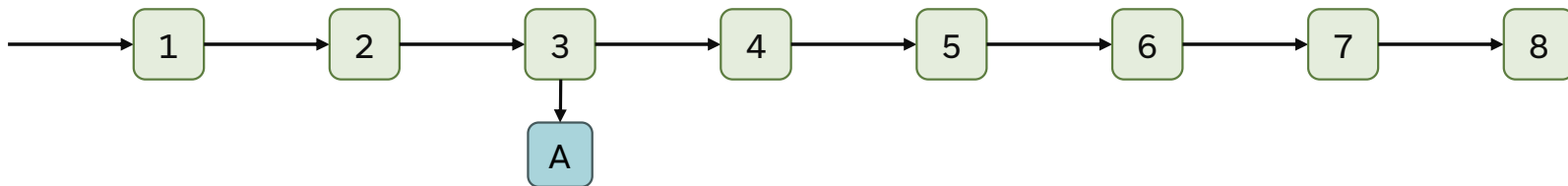
LFU Cache



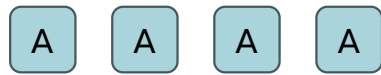
Least Frequently Used



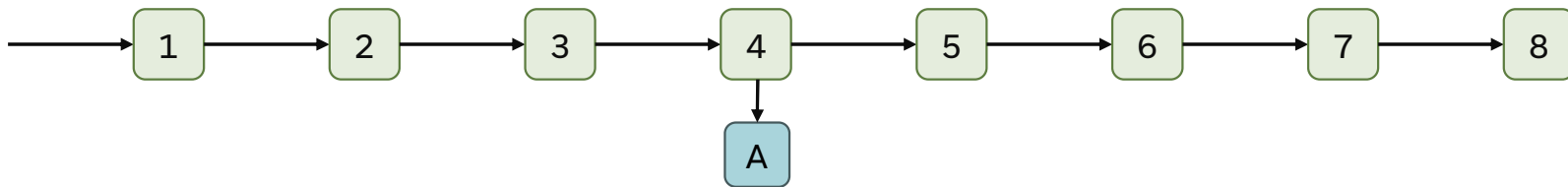
LFU Cache



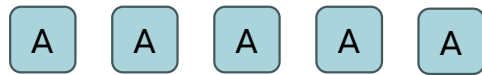
Least Frequently Used



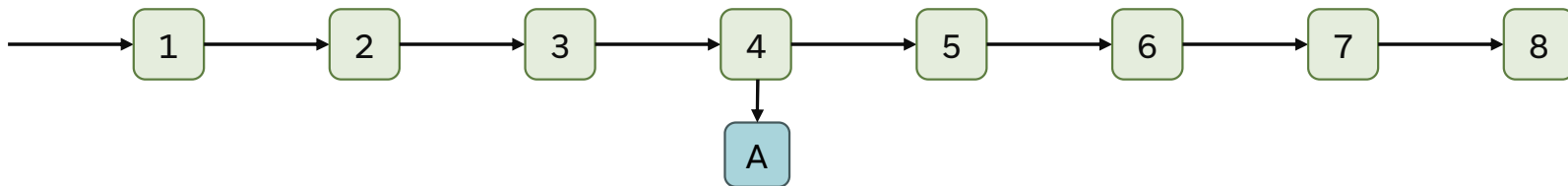
LFU Cache



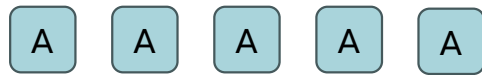
Least Frequently Used



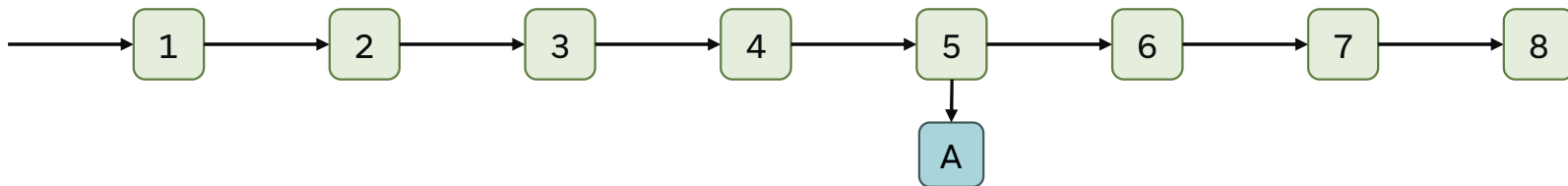
LFU Cache



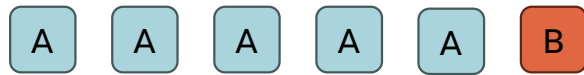
Least Frequently Used



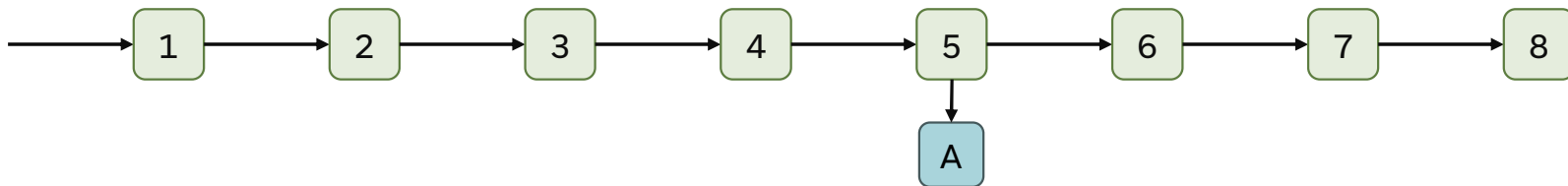
LFU Cache



Least Frequently Used



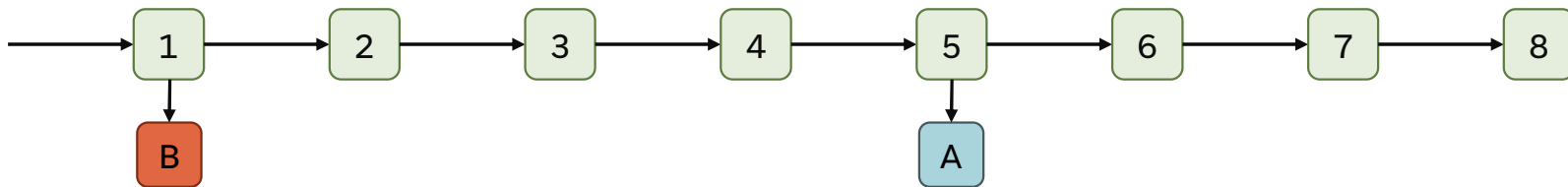
LFU Cache



Least Frequently Used



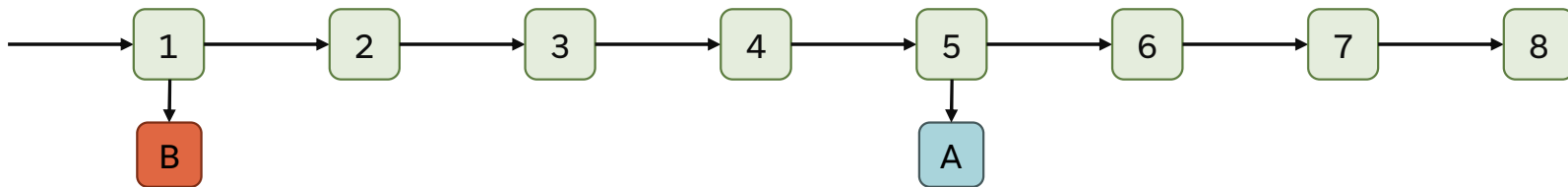
LFU Cache



Least Frequently Used



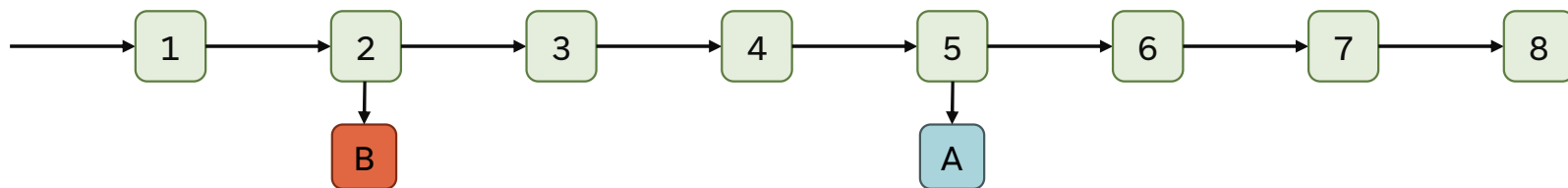
LFU Cache



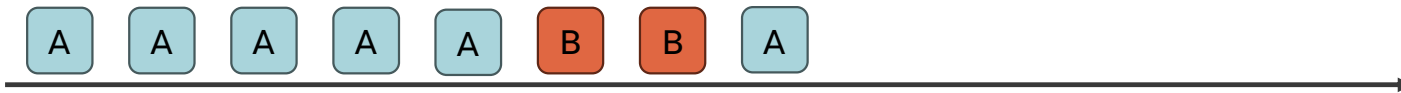
Least Frequently Used



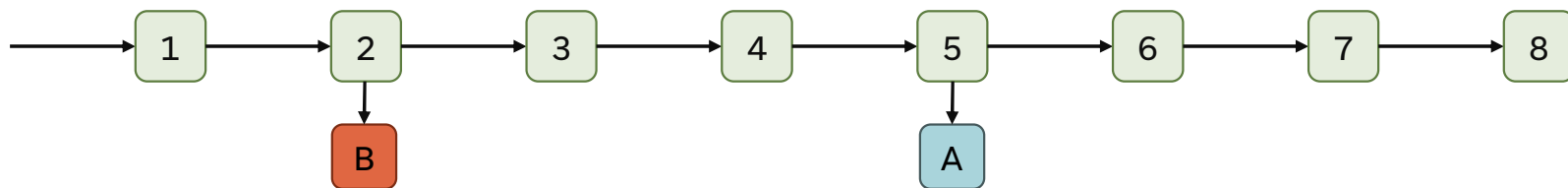
LFU Cache



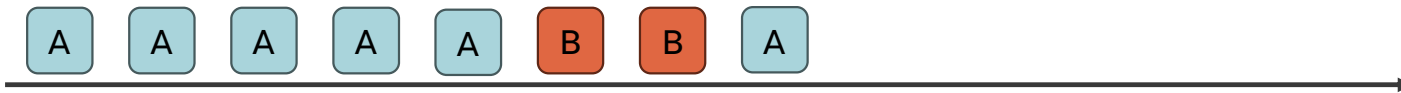
Least Frequently Used



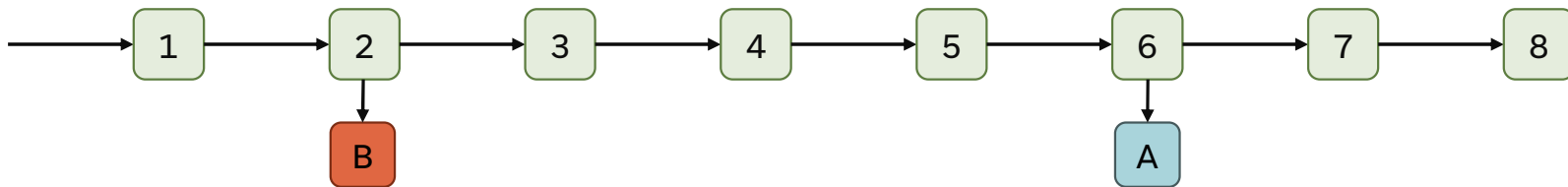
LFU Cache



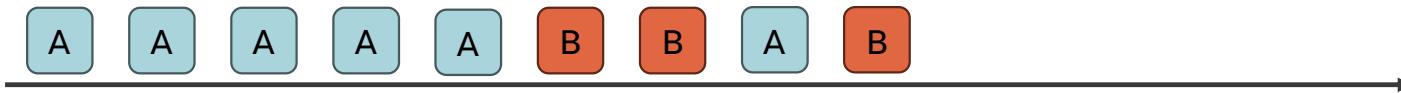
Least Frequently Used



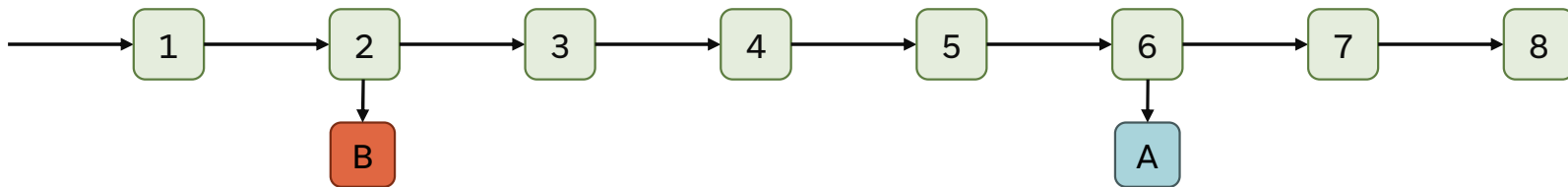
LFU Cache



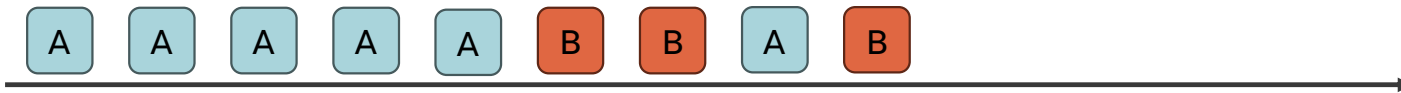
Least Frequently Used



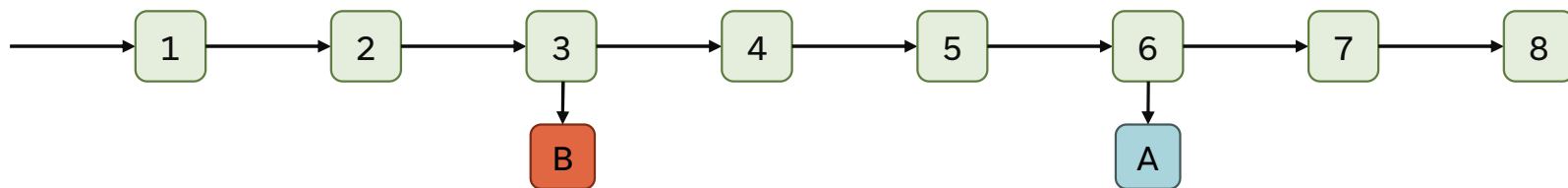
LFU Cache



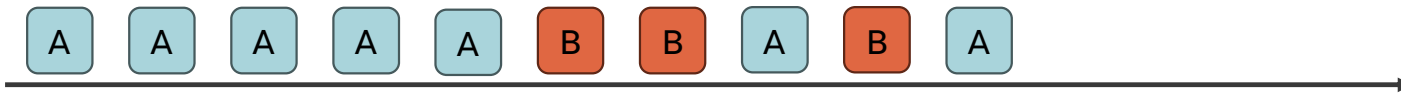
Least Frequently Used



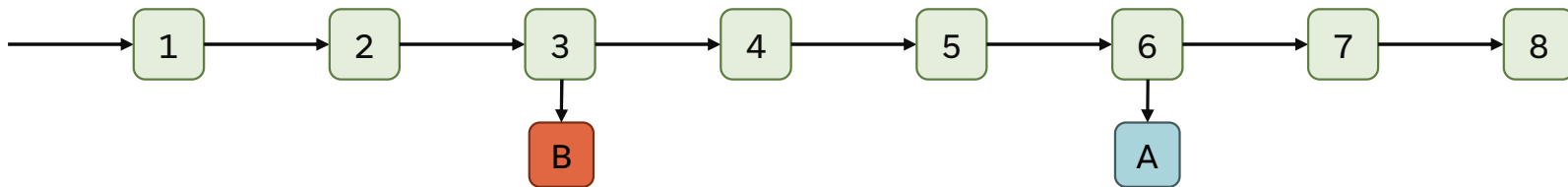
LFU Cache



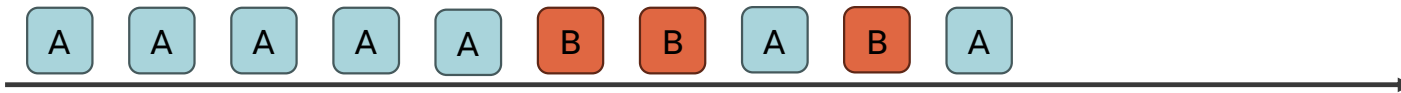
Least Frequently Used



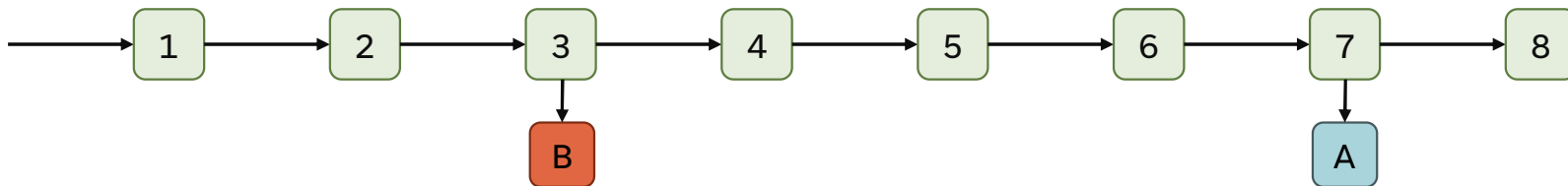
LFU Cache



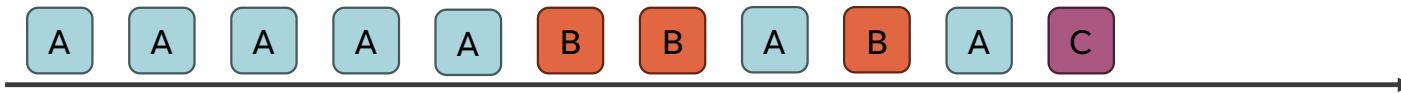
Least Frequently Used



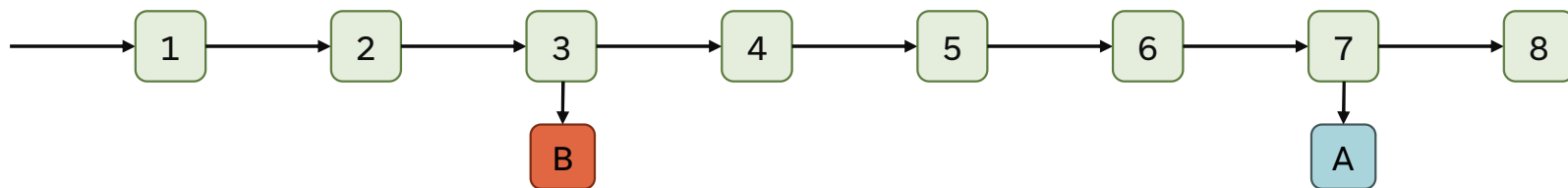
LFU Cache



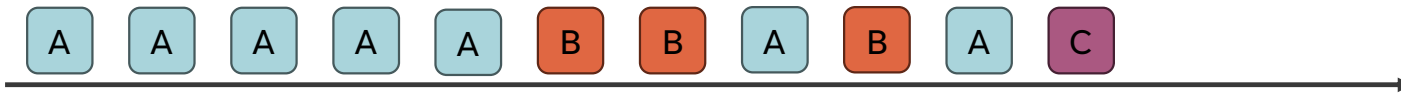
Least Frequently Used



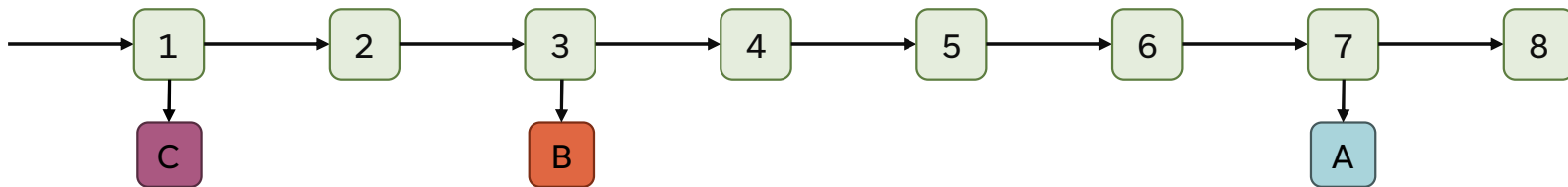
LFU Cache



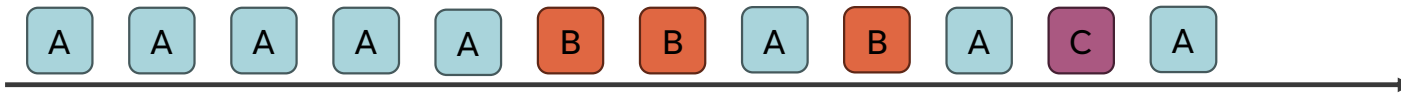
Least Frequently Used



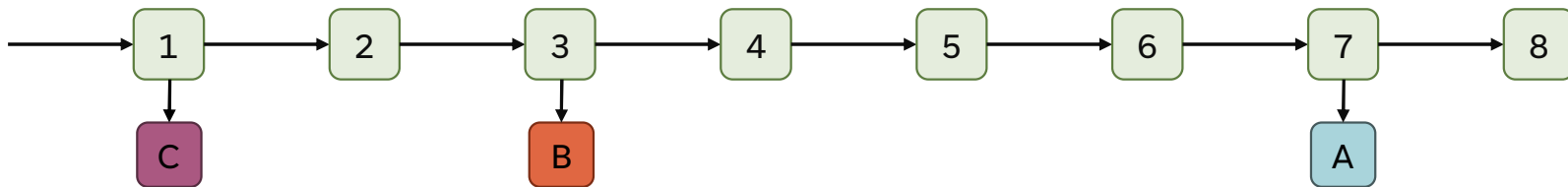
LFU Cache



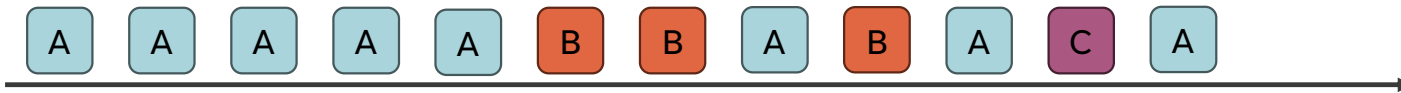
Least Frequently Used



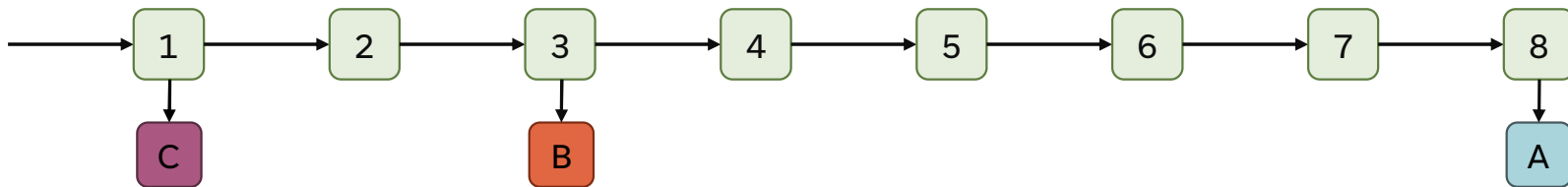
LFU Cache



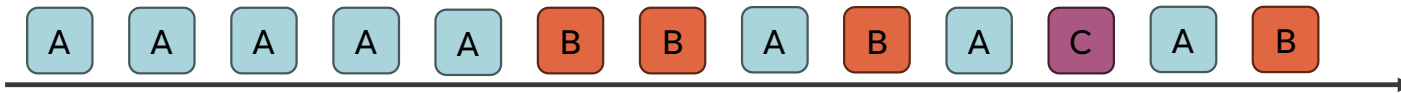
Least Frequently Used



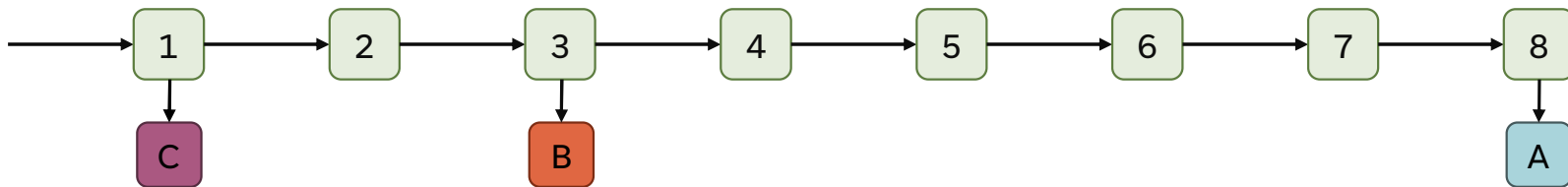
LFU Cache



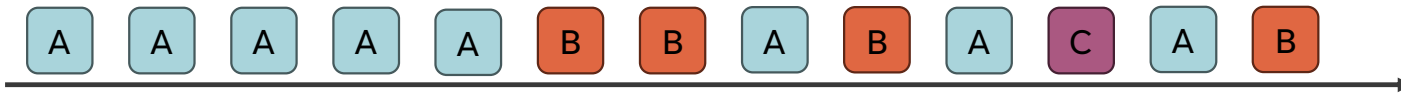
Least Frequently Used



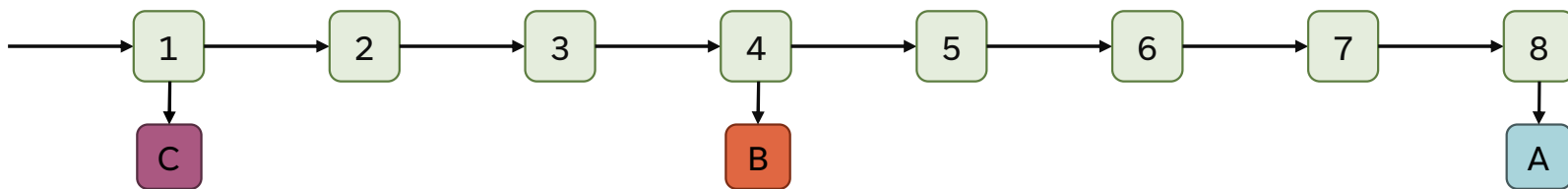
LFU Cache



Least Frequently Used



LFU Cache



Least Frequently Used

- Finding the victim can be done in $\Theta(1)$ operations using a pointer to the list containing the items with the lowest frequency.
- Excessive use of memory –
 - Many lists are empty.
 - The maximal frequency is unbounded.
 - The cache must store the meta-data of all items of the universe.
- Impractical?

Low Inter-arrival Recency Set Policy

- The *LIRS* policy is a well-known policy that observes the time-gaps between subsequent request for an item.
- The cache seeks to evict the item with the highest inter-arrival times while incorporating some form of recency.
- A very complex algorithm to implement
- Uses *Ghost-Entries*
- Used in some form in several known applications, such as MySQL



Combination of Access Patterns Policies

Segmented Least Recently Used

- The *SLRU* policy is trying to reduce the impact of One-Hit-Wonders.
- In LRU, the cache may store only One-Hit-Wonders, if a sequence longer than the cache size of such queries appears.
- Gives priority for items that appear more than once.
- Still evicts items according to their recency.

Segmented Least Recently Used

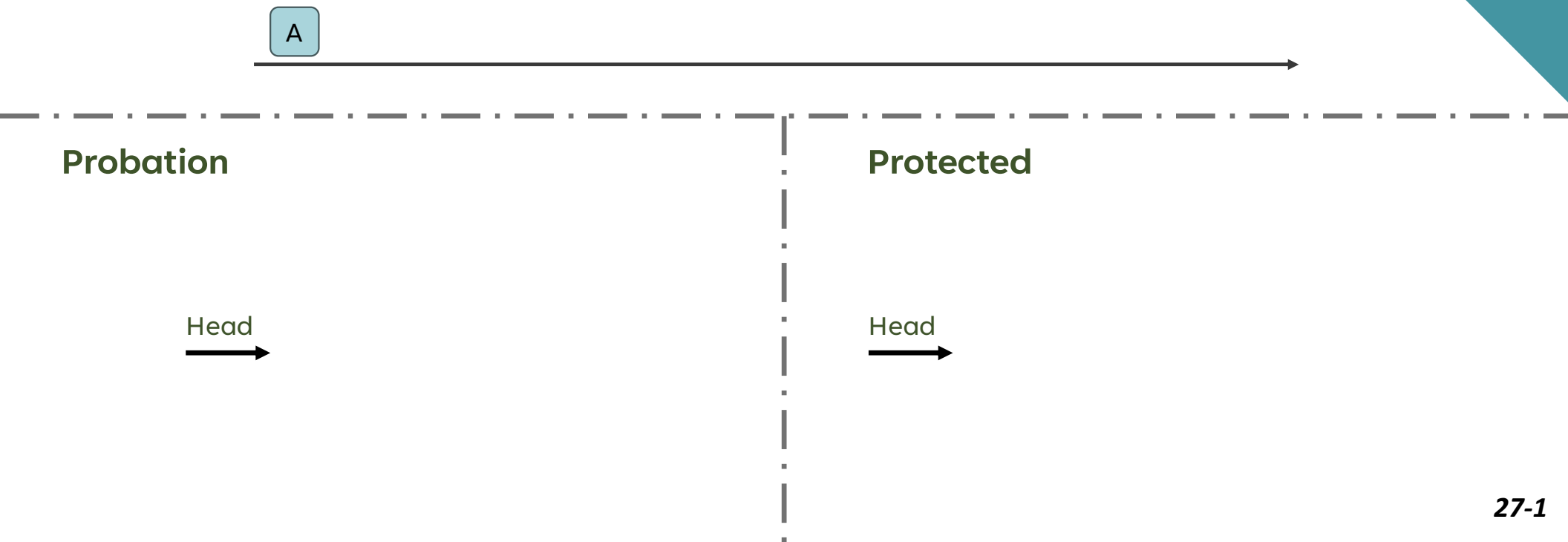
Probation

Head
→

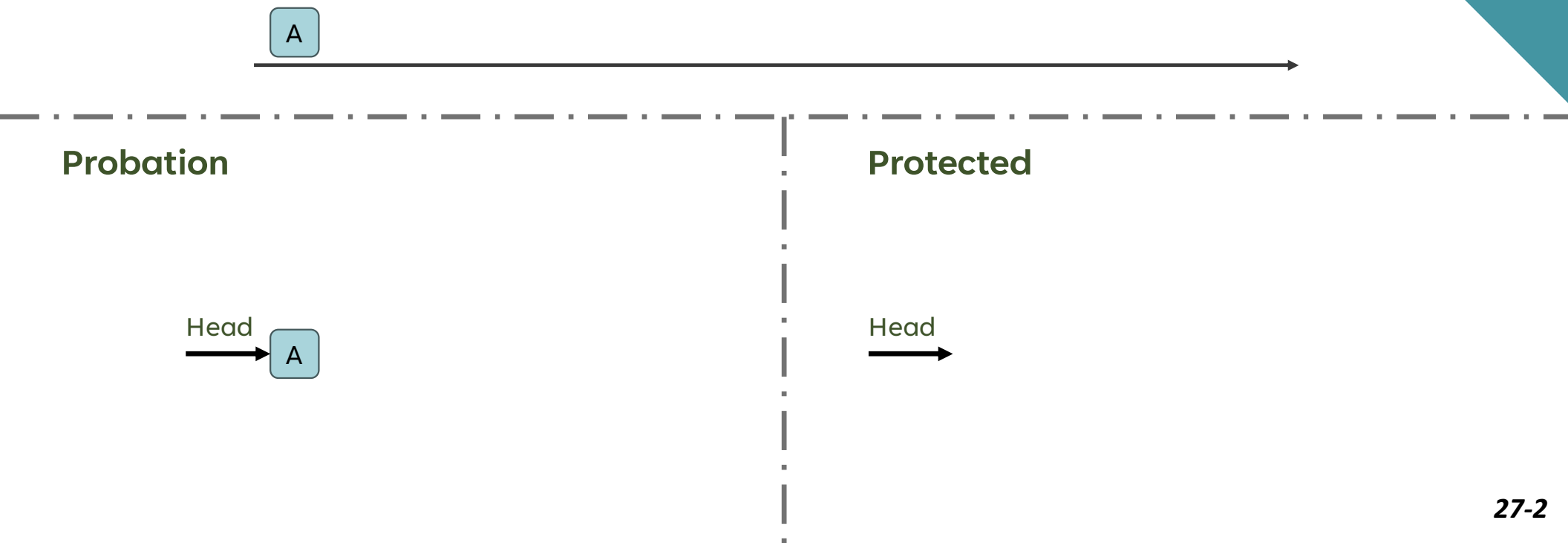
Protected

Head
→

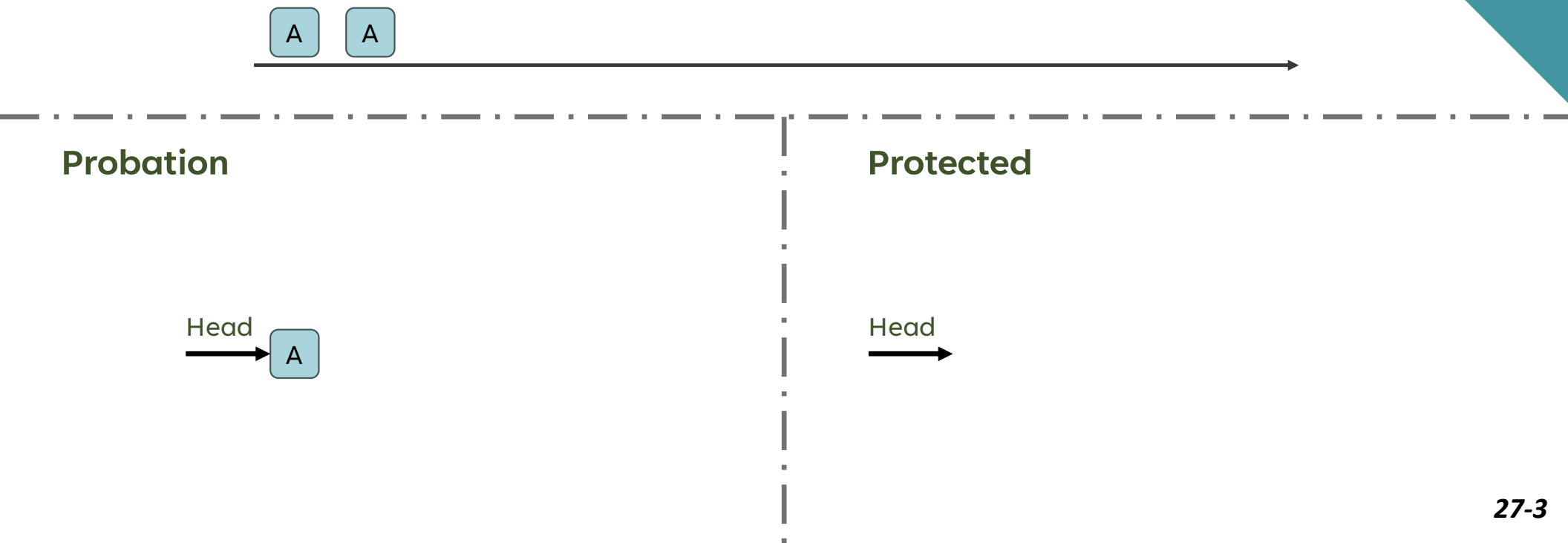
Segmented Least Recently Used



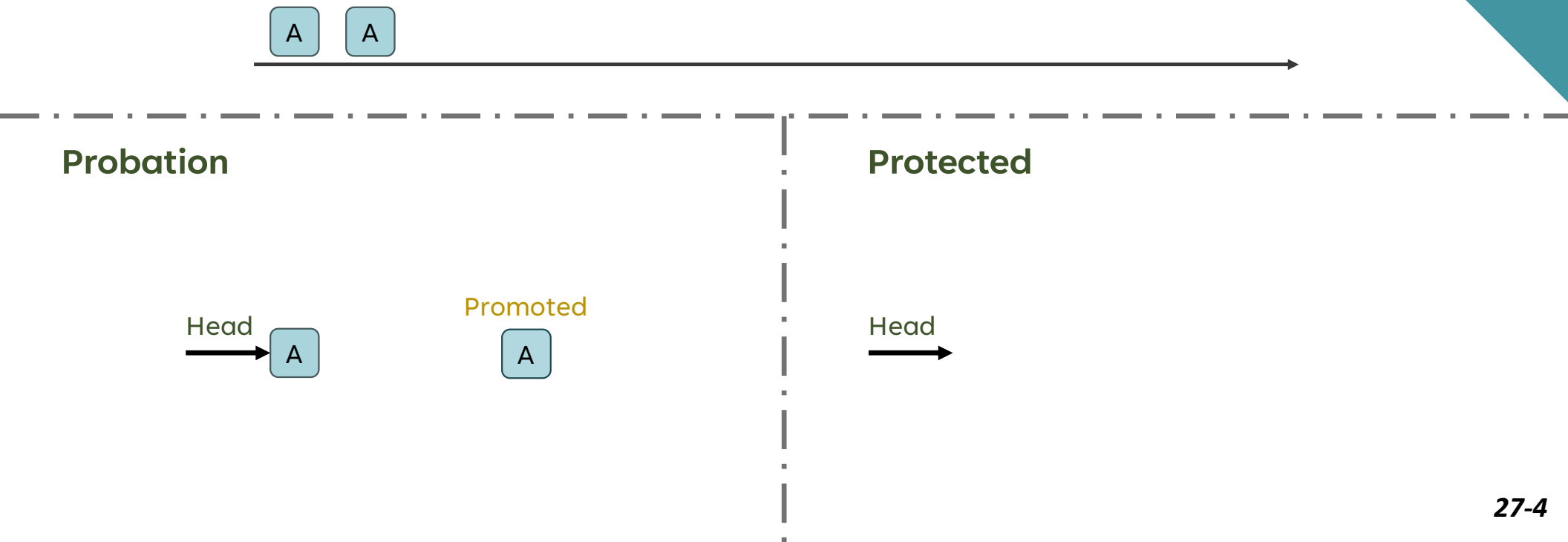
Segmented Least Recently Used



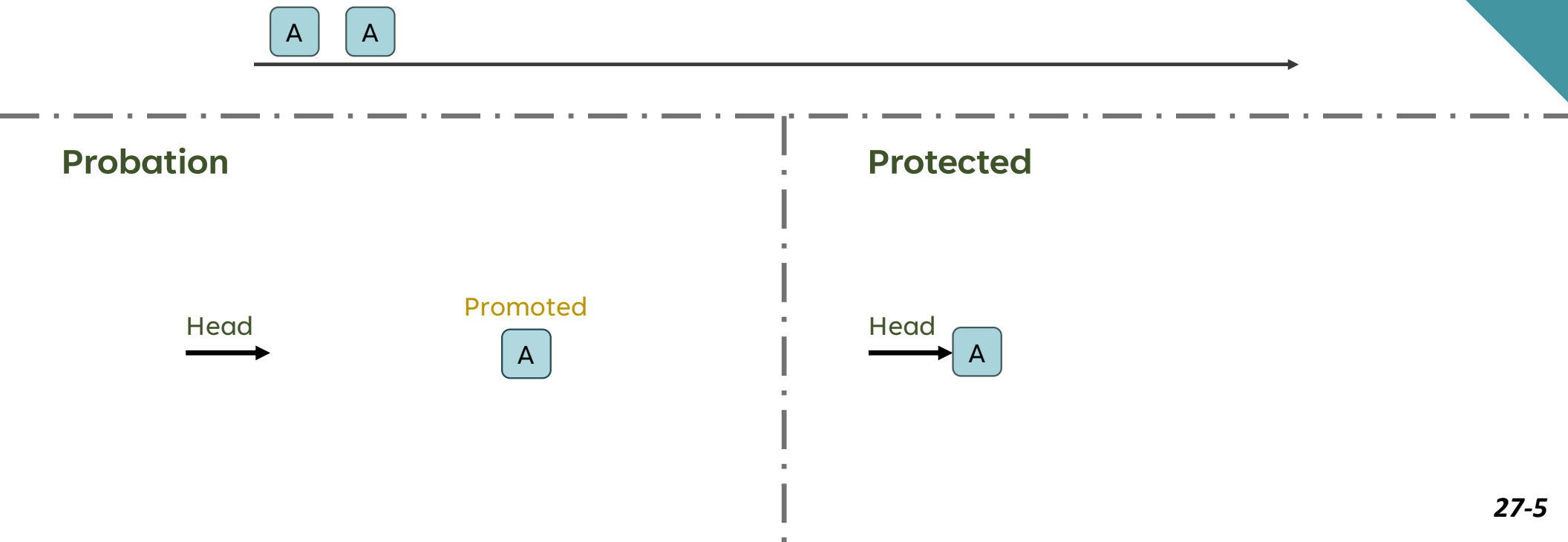
Segmented Least Recently Used



Segmented Least Recently Used



Segmented Least Recently Used



Segmented Least Recently Used



Probation

Head
→

Protected

Head
→ A

Segmented Least Recently Used



Probation

Head
→

Protected

Head
→ A

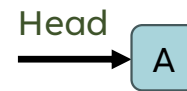
Segmented Least Recently Used



Probation



Protected



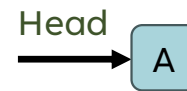
Segmented Least Recently Used



Probation



Protected



Segmented Least Recently Used



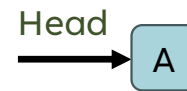
Probation



Promoted



Protected



Demoted



Segmented Least Recently Used



Probation



Promoted



Protected



Demoted



Segmented Least Recently Used



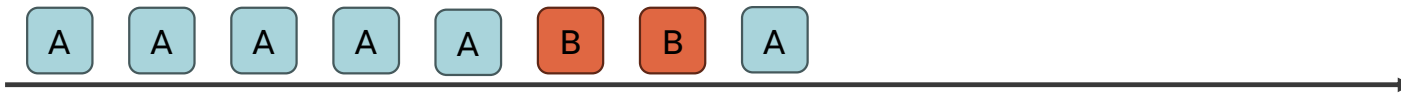
Probation



Protected



Segmented Least Recently Used



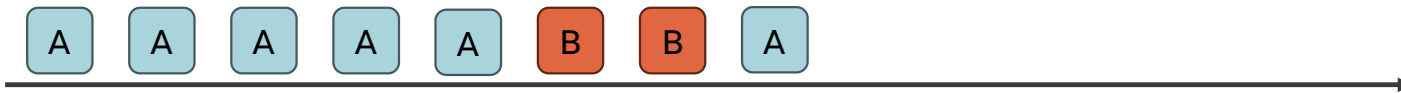
Probation



Protected



Segmented Least Recently Used



Probation



Promoted



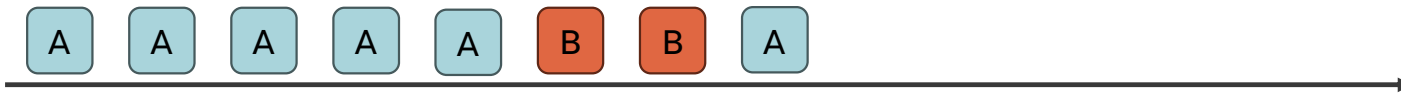
Protected



Demoted



Segmented Least Recently Used



Probation

Head

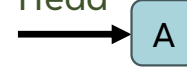


Promoted



Protected

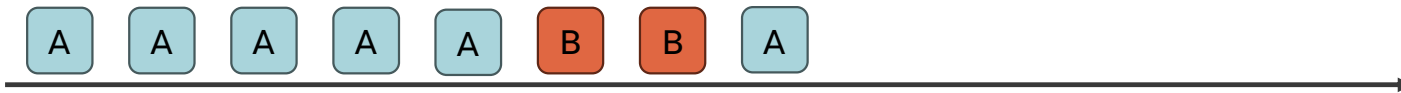
Head



Demoted



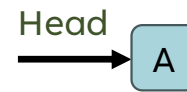
Segmented Least Recently Used



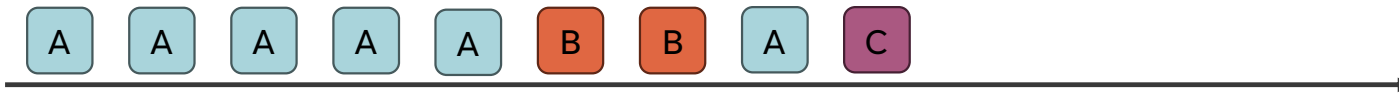
Probation



Protected



Segmented Least Recently Used



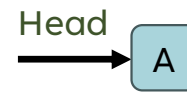
Probation



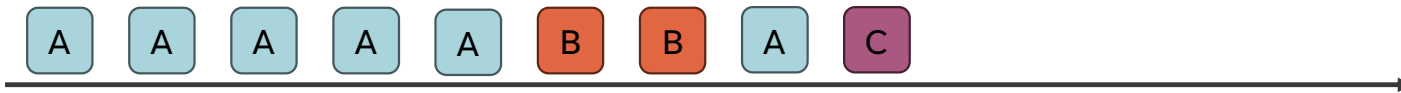
Evicted



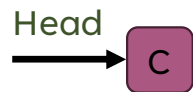
Protected



Segmented Least Recently Used



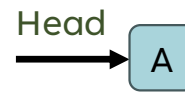
Probation



Evicted



Protected



Last Week - Recap

- What is caching?
- The classical model
- Algorithms:
 - Belady
 - FIFO
 - LRU
 - LFU
 - SLRU



Two-Queue (2Q)

- The 2Q policy works similarly to SLRU with one change:
 - The probation buffer is a FIFO policy in place of the LRU.
 - There is no demotion, items are evicted from the LRU only if the FIFO is smaller than pre-defined size.
- The idea is that the cache should evict cold pages before hot pages.
 - The FIFO holds pages yet to be proven hot.

Least Recently / Frequently Used

- A combination of both recency and frequency localities by using a complex scoring function.
- For each item b , we store the previous access times, denoted by:

$$t_1^b, \dots, t_k^b$$

- The scoring function of an item b at time t is:

$$s(b, t) = \sum_{i=1}^k f(t - t_i^b)$$

- f is a scoring function that attempts to balance between recency and frequency, such as:

$$f(x) = \left(\frac{1}{2}\right)^{\lambda x}$$

- There is a tunable parameter λ , where $\lambda \rightarrow 1$ means the cache is closer to being LFU, $\lambda \rightarrow \frac{1}{2}$ means the cache is closer to being LRU

Window Tiny-LFU

- Combines both LRU and LFU policies in one policy.
- The LFU section uses TinyLFU to estimate the frequencies of items.
 - The implementation chosen is of SLRU.
- Widely used across the industry using the Caffeine library, among those:
 - Cassandra
 - Kafka
 - Neo4j

Tiny LFU

- As mentioned before, LFU needs knowledge regarding all existing items in the universe.
 - The frequencies do not age.
- TinyLFU solves this by using Count-Min Sketch.
 - Every X operations, we age all values in the sketch by dividing all values by two (right shift).

Count-Min Sketch

F_1	F_2	F_3	F_4
	0	7	3
5	4	0	2
1	2	1	1
2	1		5
3	3	0	
0		3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

F_1	F_2	F_3	F_4
	0	7	3
5	4	0	2
1	2	1	1
2	1		5
3	3	0	
0		3	0

For each filter we have a different hash function of the type $A \cdot x + b$

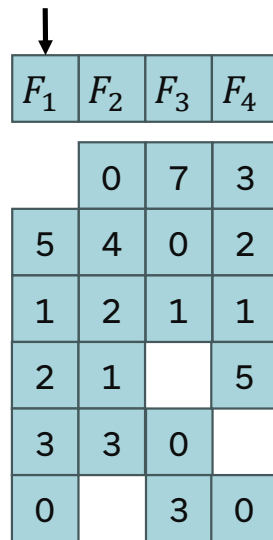
On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x



	F_1	F_2	F_3	F_4
		0	7	3
	5	4	0	2
	1	2	1	1
	2	1		5
	3	3	0	
	0		3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1		5
3	3	0	
0		3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1		5
3	3	0	
0		3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1		5
3	3	0	
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

↓

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	1
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

F_1	F_2	F_3	F_4
1	0	7	3
5	4	0	2
1	2	1	1
2	1	1	5
3	3	0	1
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- Update only the minimal cells
- Update all

On read, return the **minimal value**.

Count-Min Sketch

x

F_1	F_2	F_3	F_4
2	0	7	3
5	4	0	2
1	2	1	1
2	1	2	5
3	3	0	2
0	2	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

On write there are two options:

- **Update only the minimal cells**
- Update all

On read, return the **minimal value**.

“Conservative Update”

Count-Min Sketch

x

F_1	F_2	F_3	F_4
2	0	7	3
5	4	0	2
1	2	1	1
2	1	2	5
3	3	0	2
0	3	3	0

For each filter we have a different hash function of the type $A \cdot x + b$

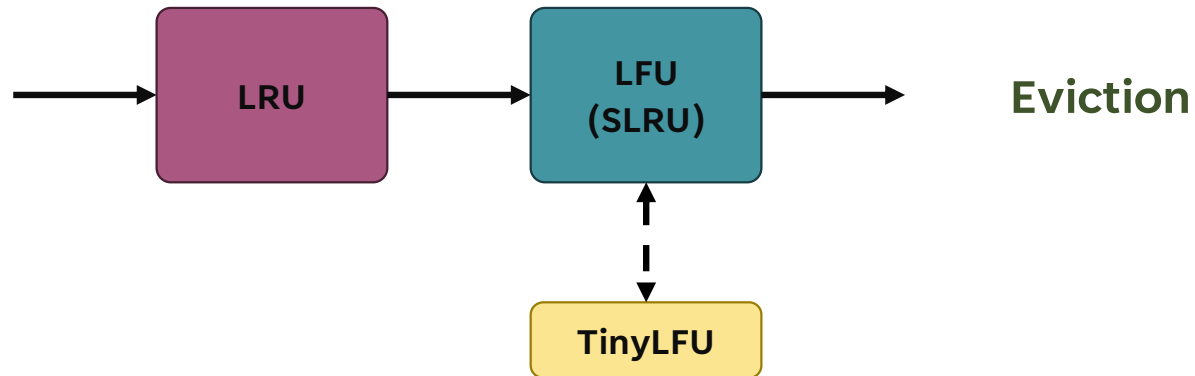
On write there are two options:

- Update only the minimal cells
- **Update all**

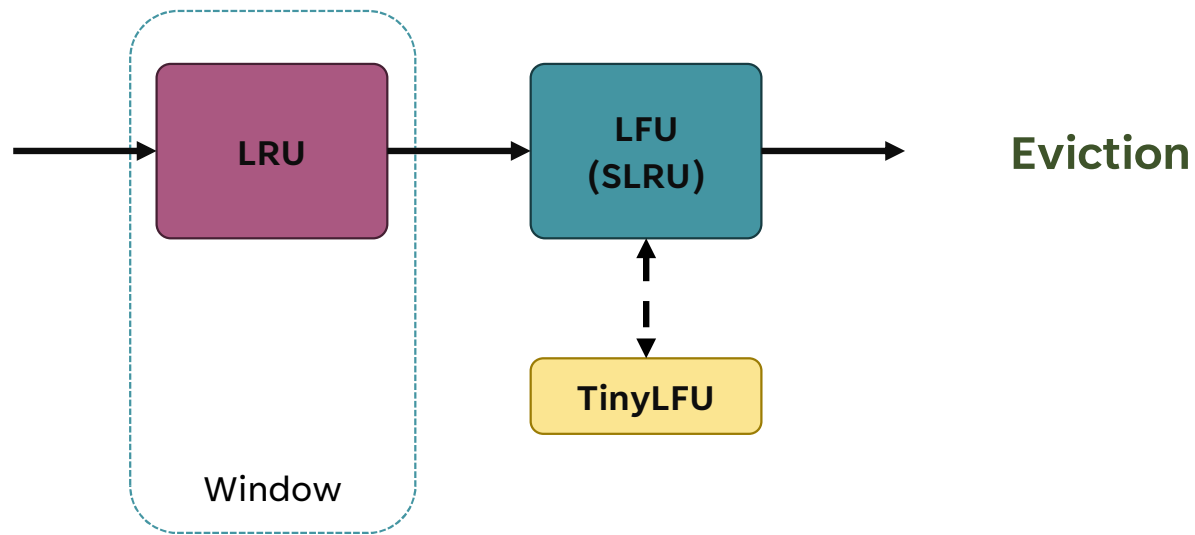
On read, return the **minimal value**.

Update All

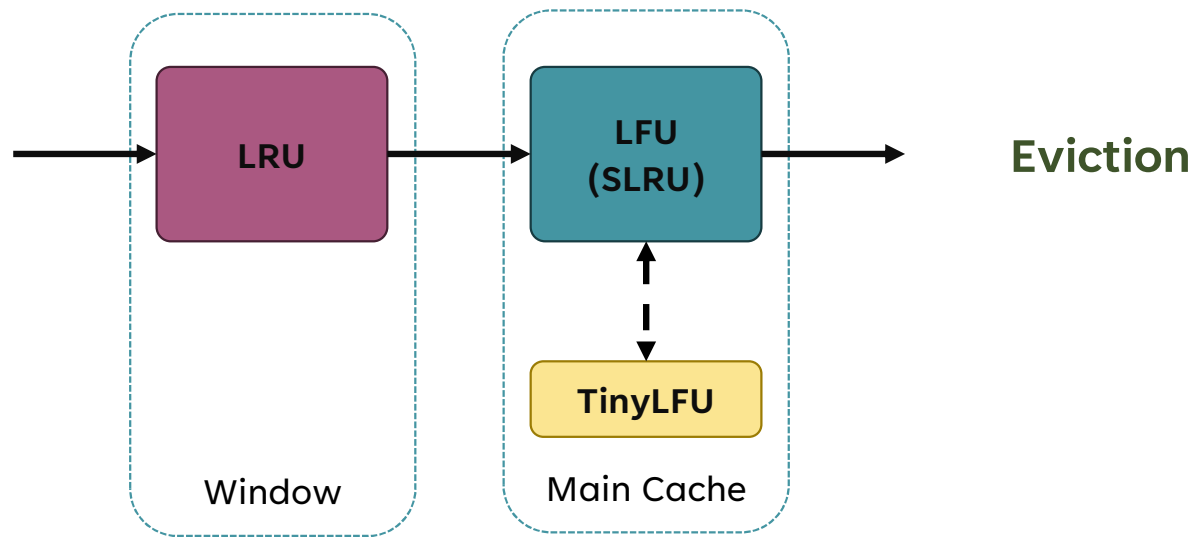
Window Tiny-LFU Architecture



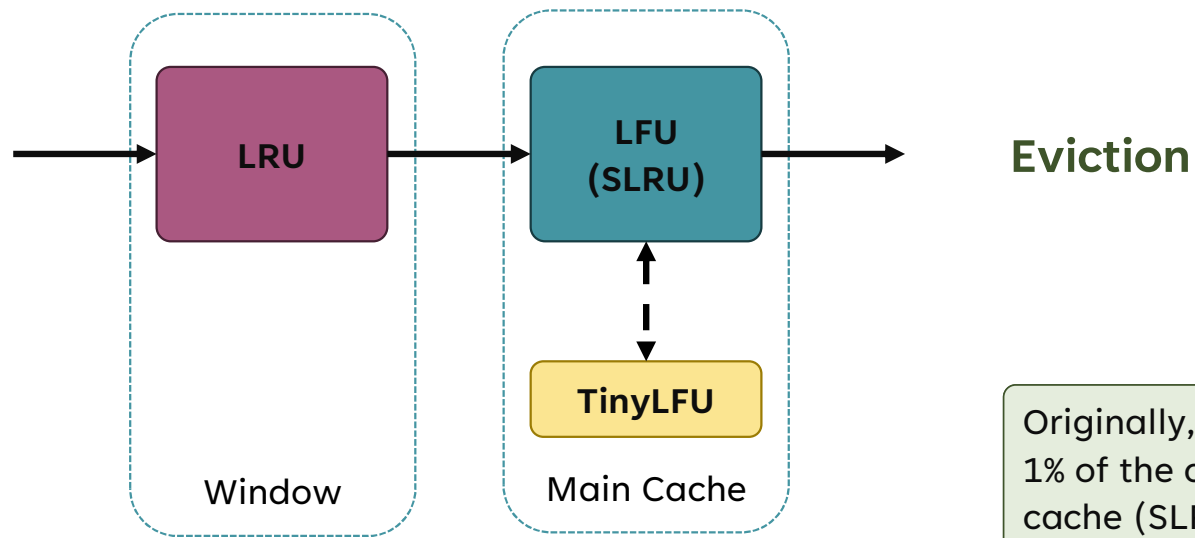
Window Tiny-LFU Architecture



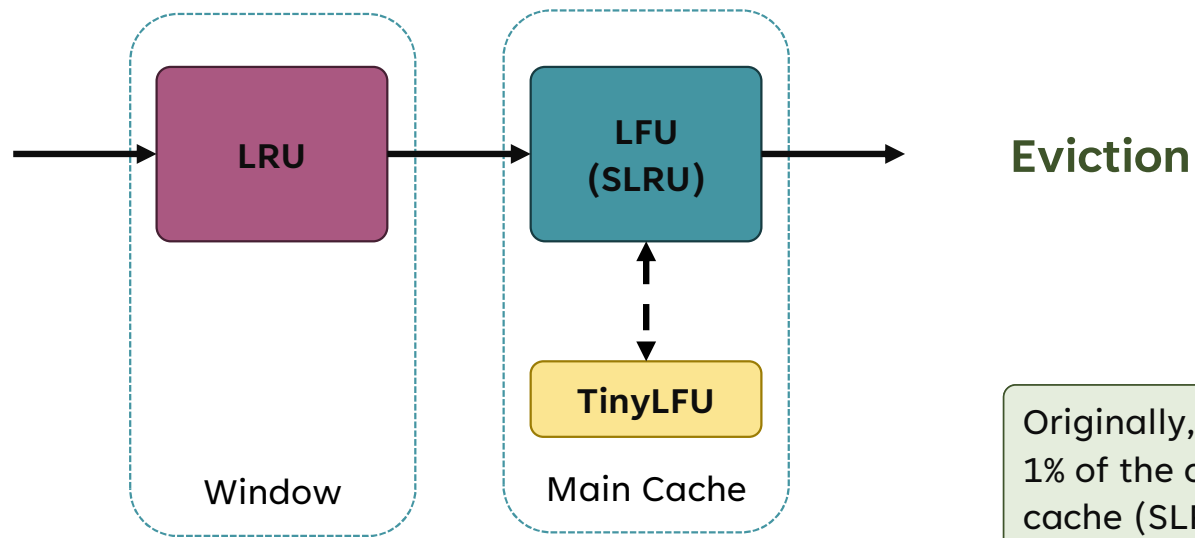
Window Tiny-LFU Architecture



Window Tiny-LFU Architecture



Window Tiny-LFU Architecture



Originally, the window (LRU) is 1% of the cache, and the main cache (SLRU + TinyLFU) is 99%

More on the size choices below!

A black and white photograph of a small seedling with two leaves growing out of dark, textured soil. The seedling is positioned on the left side of the frame, and the soil is in the foreground. The background is dark and out of focus.

Cost-Aware Caching

A sample of Cost-Aware Policies

- Hyperbolic
- Greedy-Dual
- Cost and Recency Aware Cache
- Cost-Aware W-TinyLFU



Hyperbolic

- On eviction decision:

- Randomly sample items in the cache

- Score the items according to:

$$s(i) = c_i \cdot \frac{n_i}{t_i}$$

where n_i is the occurrence count, and t_i the time since the insertion of i

- Allows adding a cost c_i to the calculations
- Evicts the lowest scored item

Greedy-Dual

- Derived from an ILP problem
- Given a request:
 - If the item i is in the cache, set its score to be c_i
 - If the item is not in the cache, evict the item with the lowest score $s_{\min}$ and update all scores to be:
$$s'_i = s_i - s_{\min}$$
insert the new item with the score c_i
 - Can be done using Min-Heap and lazy evaluations at $\Theta(\log n)$ time

Greedy-Dual



- Can be estimated in an ***amortized*** constant-time using Greedy-Dual-Wheel
- Can integrate different metrics such as:
 - Size
 - Size and Frequency
 - Weight functions
 - Keep-Alive timers

Cost and Recency Aware Cache

- The **CRA** policy combines both the recency and the cost of items.
- The last access time of each item, t_i is stored in the cache.
- The score of an item is:

$$s(i) = c_i \frac{1}{(t - t_i + 1)^k}$$

for a regularization parameter k

- Implemented using multiple LRUs grouping items of the same cost range

Cost and Recency Aware Cache

CRA

$[c_{min}, c_1)$



$[c_1, c_2)$



$[c_2, c_3)$



$[c_3, c_{max}]$



Cost-Aware Window TinyLFU

- The CA-W-TinyLFU is using CRA policy instead of the LRU blocks in both the window and the main cache, that is:
 - The window is just a CRA policy
 - The main cache is SLRU where both the probation and the protected are CRA policies.

A black and white photograph of a small seedling with two leaves growing out of dark, textured soil. The seedling is positioned on the left side of the frame, and the soil is in the foreground. The background is dark and out of focus.

Adaptive Caching

A sample of Adaptive Policies

- AdaptSize
- Darwin
- Adaptive Caching Using Multiple Experts
- Adaptive Replacement Cache
- Adaptive Window TinyLFU



AdaptSize

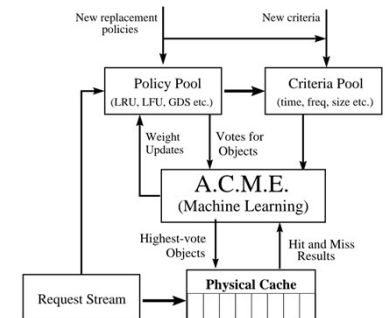
- An **admission policy** that inserts an item according to a probability.
- For an item of size s , insert it to the cache in probability:
$$p(s) = 1 - e^{-c \cdot s}$$
- Where c is an adaptive parameter of the cache.
- Learned using an approximation of a Markov Chain.

Darwin

- An Admission policy that inserts to the Hot-Ones Cache (on RAM) only items with frequency $> f$ and size $< s$.
- Uses clustering to group different traces of similar features (average item size and inter-arrival times).
- Learning **offline** what is the best (f, s) combo for each of the clusters.
- Trained Neural-Network predicts which is the most probable cluster for the current traffic, and picks the appropriate (f, s) combo.
- Uses techniques from Reinforcement-Learning (specifically, the Multi-Armed Bandit Problem).

Adaptive Caching Using Multiple Experts

- The **ACME** policy simulates multiple different static eviction policies.
- When in need of eviction, each item is voted by these multiple experts, and the lowest scored item is evicted.
- The score of an item is the weighted sum of the votes of the experts
- The votes of the experts are weighted according to a machine learning algorithm.
- Based on Reinforcement Learning.



Adaptive Replacement Cache

- The **ARC** policy is a policy balancing between storing items that appear only once and items that appeared multiple times.
- Uses two LRU policies, named T_1 and T_2 to store both item types with adaptive size.
 - The combined size is fixed, that is:
$$|T_1| + |T_2| = \text{cache size}$$
 - Using ghost entries to determine the size split between the two, in LRU policies B_1 and B_2
 - The ghosts complete T_1 and T_2 to the total cache size.

Adaptive Replacement Cache

OnHit(item i):

if i in T_1 or T_2 :

move i to be the head of T_2

if i in B_1 :

insert i as the head of T_1

increase the size of T_1 on the account of T_2 :

$p \leftarrow p - 1$

Evict()

if i in B_2 :

insert i as the head of T_2

increase the size of T_2 on the account of T_1 :

$p \leftarrow p + 1$

Evict()

OnMiss(item i):

insert i as the head of T_1

Evict()

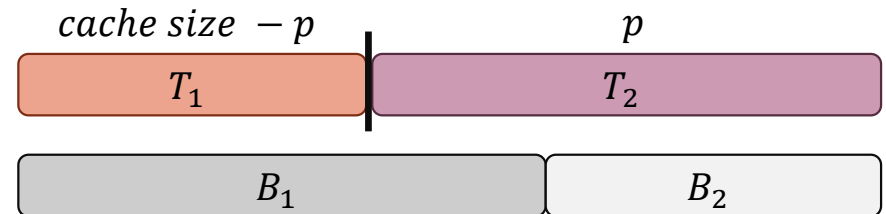
Evict():

if $size(T_2) > p$:

evict the LRU of T_2

else:

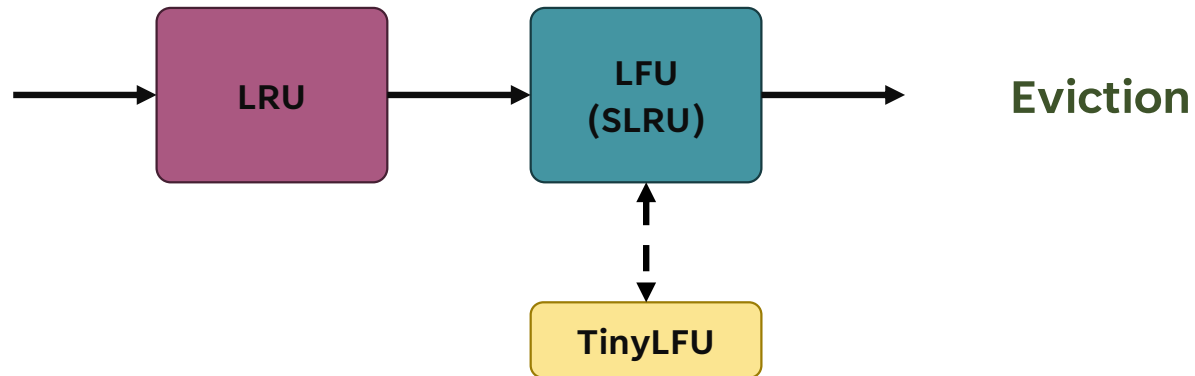
evict the LRU of T_1



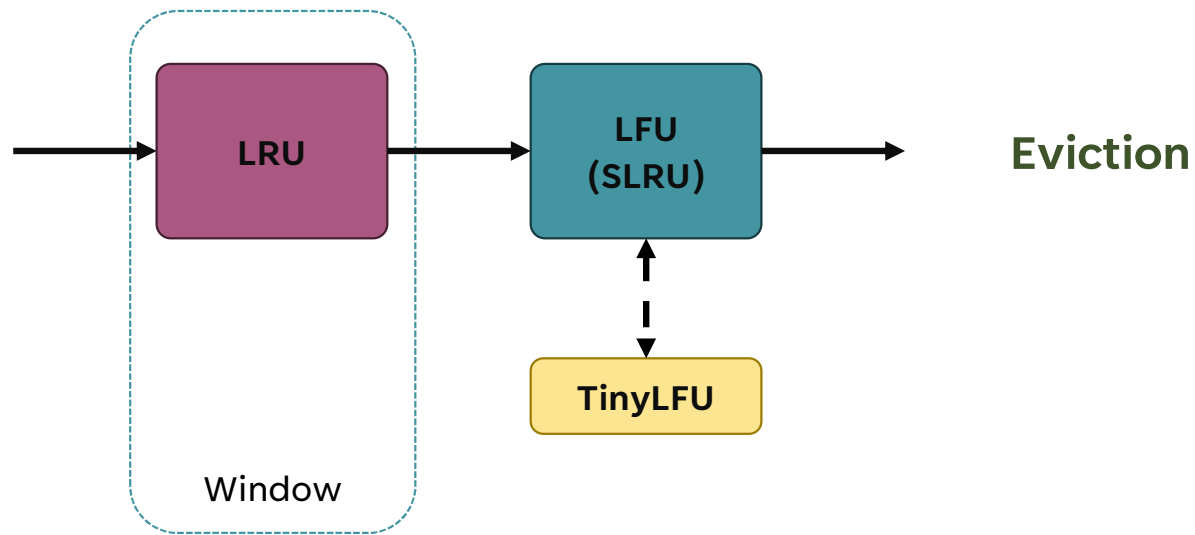
Adaptive Replacement Cache

- The main idea is:
 - What would have happened if T_1 (or T_2) was larger?

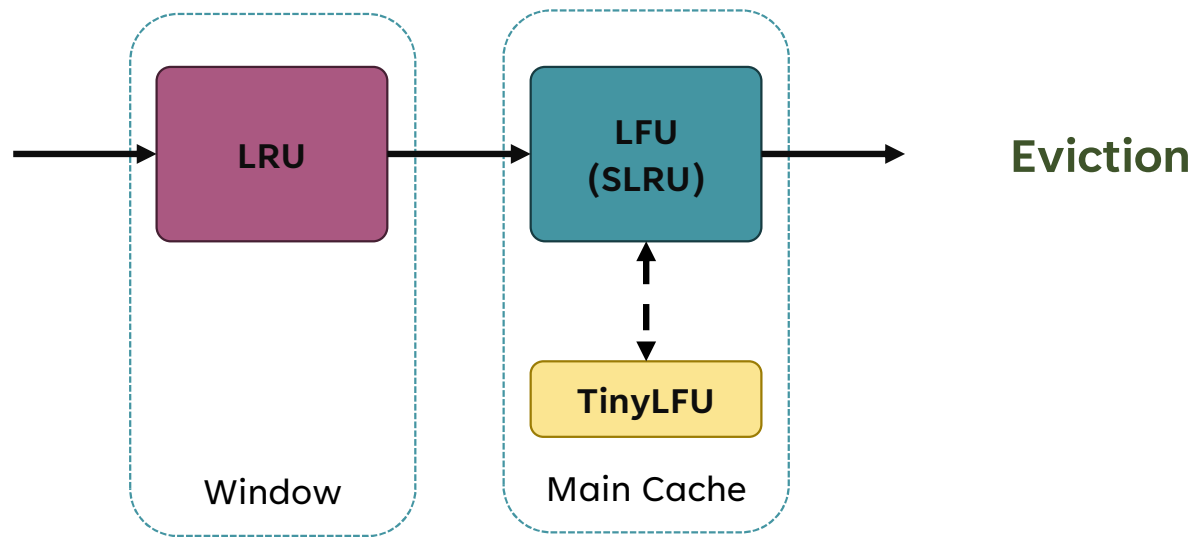
Adaptive Window Tiny-LFU



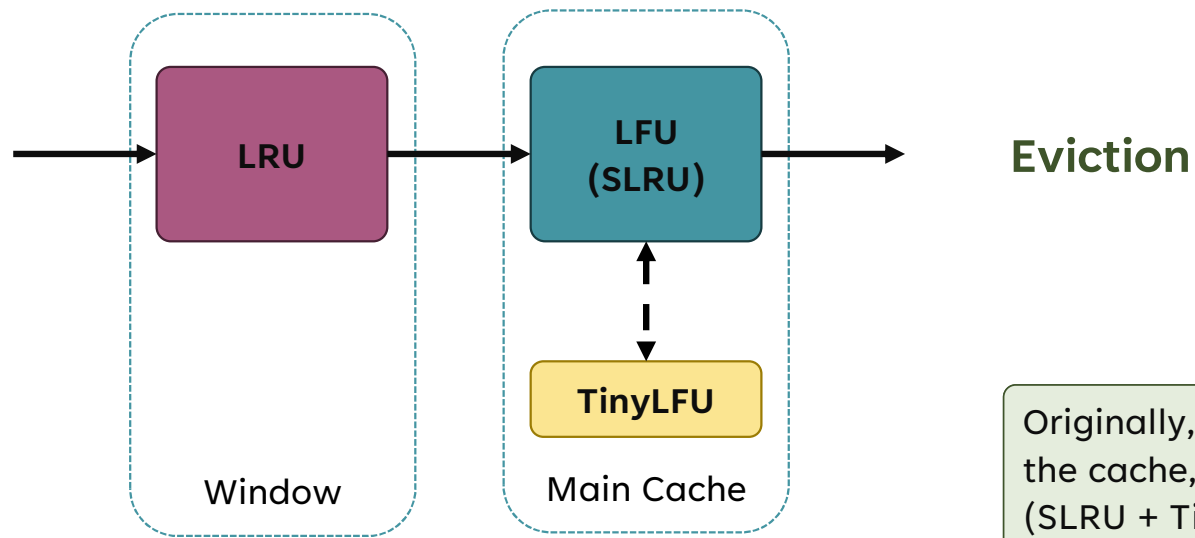
Adaptive Window Tiny-LFU



Adaptive Window Tiny-LFU

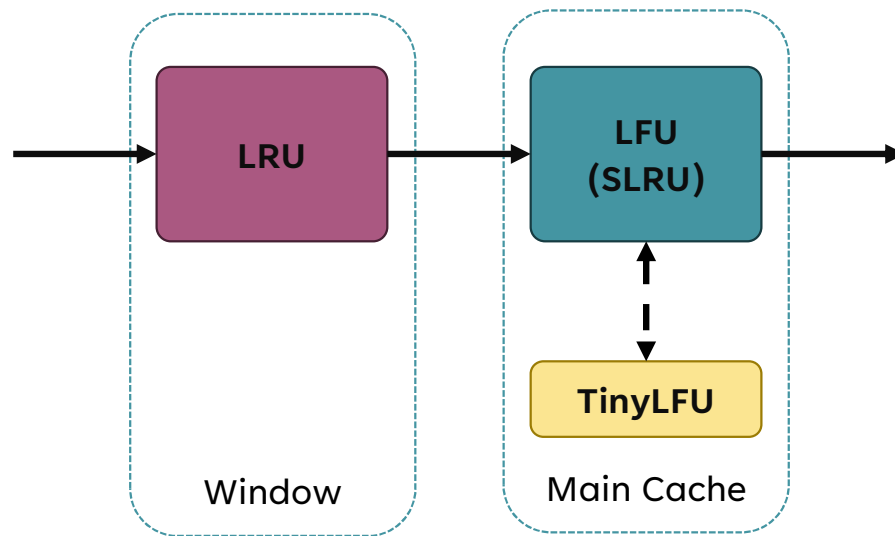


Adaptive Window Tiny-LFU



Originally, the window (LRU) is 1% of the cache, and the main cache (SLRU + TinyLFU) is 99%

Adaptive Window Tiny-LFU



Eviction

Originally, the window (LRU) is 1% of the cache, and the main cache (SLRU + TinyLFU) is 99%

Every timeframe, 5% is transferred between the sections